

Full Length Article

Retrieval-augmented and feedback-optimized large language model for recommendation

Zhisheng Yang , Li Li *

School of Computer and Information Science, Southwest University, 400715, Chongqing, China

ARTICLE INFO

Keywords:

Large language models
Retrieval-augmented generation
Kdtree
Non-tuning
Collaborative filtering

ABSTRACT

Large Language Models (LLMs), with their powerful generative capabilities, have given rise to the emerging paradigm of “LLM-as-Recommend Systems” (LLM-as-RS). However, challenges such as data noise, the static nature of LLMs, and input length limitations hinder their full potential and make model updates costly. To address these issues, this study proposes an innovative generative recommendation framework: KDRAG-Critic-LLM-RS, which integrates retrieval augmentation and feedback optimization. Specifically, we design an efficient KDTreeRAG retrieval module, based on the principle of user-based collaborative filtering, to rapidly identify similar users and provide structured behavioral context. In addition, we introduce a lightweight R-Critic feedback module, inspired by item-based collaborative filtering, which captures user feedback signals to refine the generated recommendations and improve adaptability. By establishing a closed-loop mechanism of retrieval-generation-feedback, this framework effectively integrates collaborative signals from both user and item perspectives, continuously optimizing recommendation quality. Extensive experiments on real-world movie and book datasets demonstrate that KDRAG-Critic-LLM-RS significantly outperforms traditional baselines in terms of accuracy, personalization, and diversity—without requiring any fine-tuning of the LLM. More importantly, the framework is modular and plug-and-play, allowing the retrieval and feedback components to be easily integrated into any LLM-based recommendation system. This design avoids the high cost of frequent LLM retraining, mitigates the impact of noisy data, and overcomes input length limitations, ultimately enhancing the system's responsiveness to dynamic user preferences and improving generalization.

1. Introduction

In recent years, the powerful generative and reasoning capabilities of Large Language Models (LLMs) have driven a shift in recommender systems from traditional collaborative filtering to generative recommendation paradigms (Wang et al., 2024; Wu et al., 2024). Leveraging the natural language generation and abstraction abilities of LLMs, the “LLM-as-RS” paradigm enables end-to-end production of recommendation lists without relying on fixed item embeddings or dense interaction matrices.

Although recent LLM-based recommenders inject collaborative-filtering (CF) signals, many still rely on static pre-trained retrievers or fixed graph priors, making them brittle to dynamic shifts in user behavior (Bei et al., 2025; Huang et al., 2025a; Zhang et al., 2025c). Moreover, they rarely optimize retrieval with generative feedback, limiting adaptation to real-time drift, data noise, cold-start items, and evolving preferences (Zhang et al., 2025d). To address these gaps, we propose KDRAG-Critic-LLM-RS, which couples a user-CF, retrieval-enhanced generator (KDTreeRAG) with an item-CF, feedback-driven opti-

mizer (R-Critic). KDTreeRAG performs efficient KDTree neighbor search to steer candidate generation, while R-Critic turns predicted rating signals into *iterative feedback* that updates the retrieval-generation process. This closed-loop retrieve→generate→feedback mechanism mines CF signals along both user and item axes and adapts them dynamically, improving recommendation accuracy and diversity under drift and cold-start conditions (Huang et al., 2025a; Zhang et al., 2025d).

Common generative baselines (e.g., ChatGPT, GPTRec, FLAN-T5, TALLRec, ZeroRec) typically produce recommendations via prompt engineering or by pre-training on user interaction histories for in-context learning (Harrison et al., 2023; Kang et al., 2023; Kocón et al., 2023). While these approaches are flexible and strong in zero-shot settings, their lack of explicit user-item modeling often leads to degraded performance in data-sparse or cold-start scenarios (Huang et al., 2025a; Liu et al., 2025; Shi et al., 2025; Zhang et al., 2025d).

To improve personalization, another line of work injects structured external signals (e.g., user profiles, item metadata, historical behaviors) into generation, implicitly incorporating synergy during prompting or

* Corresponding author.

E-mail addresses: yzsheng@email.swu.edu.cn (Z. Yang), lily@swu.edu.cn (L. Li).<https://doi.org/10.1016/j.neunet.2026.108954>

Received 10 July 2025; Received in revised form 1 April 2026; Accepted 4 April 2026

Available online 7 April 2026

0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

fine-tuning. Representative approaches include RecSysLLM, GENREC, IDGenRec, Prompt4Rec, BegoniaGPT, PAP-Rec, and InteraRec (Chu et al., 2023; Karra & Tulabandhula, 2024; Li et al., 2023a; Rong et al., 2025; Tan et al., 2024; Xu et al., 2024; Zhu et al., 2024). A complementary line explicitly incorporates CF by integrating user- or item-level similarities into LLM frameworks: (i) user-CF (RecMind, CoSeRec, UniRec) (Boz et al., 2024; Gong et al., 2023; Liu et al., 2024); (ii) item-CF (GENREC, LLM-TRSR) (Zheng et al., 2024b; Zhu et al., 2024); and (iii) joint user- and item-CF via graph modeling (EAGER-LLM, GaCLLM, RLMRec, RS-Agent, InteraRec) (Bei et al., 2025; Du et al., 2024; Hong et al., 2025; Karra & Tulabandhula, 2024; Ren et al., 2024; Xu et al., 2024). However, many of these systems still depend on pre-trained retrievers or static graph structures and lack a generative-feedback mechanism to *optimize retrieval* online, reducing responsiveness to behavioral drift and limiting robustness in cold-start settings.

Our approach operationalizes this gap with a plug-and-play framework. KDRAG-Critic-LLM-RS enriches generation contexts via efficient user-CF retrieval (KDTreeRAG) and leverages item-side interactions to produce feedback signals that refine generated candidates (R-Critic). Unlike static KDTree usage or fixed graph priors, KDTreeRAG *conditions token-level generation* and is *refined online* by R-Critic within a retrieve→generate→feedback loop, rather than acting as a detached retriever. The result is a dynamic, closed loop that enhances personalization and accuracy while mitigating noise-induced bias and improving robustness to preference drift.

The main contributions of this work are summarized as follows:

- We introduce *KDRAG-Critic-LLM-RS*, a plug-and-play closed-loop retrieve→generate→feedback framework that unifies user-CF retrieval and item-CF feedback for LLM-based recommendation, and can be attached to large *black-box* models *without fine-tuning or modifying their parameters*.
- We design *KDTreeRAG*, a fast user-neighbor retriever that leverages KDTree search to balance accuracy and efficiency, and *R-Critic*, a rating-based *regressor* that converts continuous predicted scores into concise textual guidance to iteratively refine LLM-generated candidates.
- We provide an end-to-end training/evaluation recipe that *optimizes the KDTreeRAG retriever* via generative feedback (online retrieval refinement) *without updating the LLM*, improving adaptability to behavioral drift, data noise, and cold-start scenarios (Zhang et al., 2025d).
- Extensive experiments on movie and book benchmarks show consistent gains in Precision/HR/NDCG over prompt-only and prior retrieval/graph-based LLM recommenders, while preserving practical inference efficiency and enhancing personalization in sparse, noisy, and drifting environments.

In summary, this work centers on a pluggable retrieve-generate-feedback loop that jointly mines collaborative signals on both the user and item sides, and achieves robust adaptation to behavioral drift and cold-start scenarios via dynamic feedback. The remainder of the paper presents the framework design and implementation, the experimental setup and evaluation protocol, and results with analyses on multiple benchmarks.

2. Related work

2.1. LLMs for recommendations

In recent years, Large Language Models (LLMs)—with strong reasoning and generation—have driven a shift in recommender systems from traditional structured modeling to a semantic, generative paradigm (Lin et al., 2025; Wang et al., 2024; Wu et al., 2024). Research on LLM-based recommendation can be organized by whether and how collaborative filtering (CF) signals are introduced, yielding three families: *user-side CF*, *item-side CF*, and *dual CF* that fuses both user and item signals (Shen et al., 2025).

Purely semantic LLM recommenders. Some approaches rely primarily on the semantic capabilities obtained during LLM pre-training. For example, ChatGPT-, GPTRec-, and LLaMA-based recommenders model user preferences via prompt engineering, in-context learning, or fine-tuning without explicitly encoding structural similarities among users or items (Kocor et al., 2023; Liu et al., 2023; Petrov & Macdonald, 2023). Building on this direction, TALLRec, E4SRec, and FLAN-T5 improve quality through instruction fine-tuning (Bao et al., 2023; Kang et al., 2023; Li et al., 2023b), while ZeroRec, AutoRec, and DrdT emphasize zero-shot capabilities in cold-start scenarios (Dai et al., 2023; Harrison et al., 2023; Wang et al., 2023). Although flexible and generalizable, these methods often underperform in deeply mining user preferences and improving accuracy due to the lack of explicit user-item interaction modeling.

Implicitly injecting structure via prompts/inputs. A second line improves accuracy by feeding user or item characteristics as cues, implicitly injecting co-occurrence information into the generation process to enrich context. For example, RecSysLLM, GENREC, and IDGenRec guide LLMs by injecting item attributes or co-occurring structures (Chu et al., 2023; Tan et al., 2024; Zhu et al., 2024); Prompt4Rec, PMG, and PAP-Rec further optimize prompt design to enhance contextual consistency and personalization (Li et al., 2023a, 2024b; Shen et al., 2024). In addition, NewRec and DrdT introduce user behavioral sequences to partially exploit user-side collaborative information, yet they still lack *explicit* modeling of user similarities (Che et al., 2024; Wang et al., 2023). Beyond generic prompts, *AutoDisenSeq-LLM* employs LLM-guided disentanglement to better capture multi-interest behavioral sequences, improving sequence-aware generation without explicit CF graphs (Wang et al., 2025). Moreover, *user-controllable profiles* expose editable, human-understandable preference fields that can be injected as structured prompts to steer LLMs toward personalized outputs (Woźniak et al., 2025).

Explicit CF with LLMs. To better exploit LLMs' generative capability while improving accuracy, a growing body of work explicitly introduces CF mechanisms. These generally fall into three paradigms:

User-based CF. Methods construct a user-similarity structure (often with contrastive learning) to guide LLMs toward personalized recommendations. RecMind conditions on similar users' histories to improve cold-start recommendations (Boz et al., 2024); CoSeRec enhances the transferability of user representations via contrastive learning (Liu et al., 2024); UniRec improves user-side collaborative generalization (Gong et al., 2023). However, these approaches may underutilize item-side synergistic relations.

Item-based CF. Methods capture structural/semantic relations between items via similarity or attribute embeddings. GENREC builds explicit item collaboration graphs to improve generation accuracy (Zhu et al., 2024); LLM-TRSR exploits item-association structures to strengthen semantic consistency and boost performance (Zheng et al., 2024b). These methods often improve diversity but remain limited in the breadth of collaborative signals modeled.

Dual CF fusion. Methods jointly model user- and item-side signals. EAGER-LLM encodes user and item graphs with GNNs to produce recommendations (Hong et al., 2025); GaCLLM introduces bipartite structural information for better generalization under cold start (Du et al., 2024); RLMRec fuses user-item behavioral sequences with structural embeddings to achieve collaboration-driven personalization (Ren et al., 2024). In addition, RS-Agent and InteraRec further enhance performance through multi-module architectures or multimodal knowledge modeling (Karra & Tulabandhula, 2024; Xu et al., 2024).

Orthogonal robustness lines (bias mitigation & deconfounding). Complementary to the above semantic-CF spectrum, recent studies target statistical bias and confounding in (LLM-)recommendation. **CLLMR** mitigates propensity bias in LLM-RS via causal/counterfactual alignment of LLM-generated side information with collaborative signals

(Zhang et al., 2025b). Beyond LLM-specific methods, representation-level debiasing with **instrumental variables (IV)** and **deconfounding contrastive learning** reduces spurious correlations and latent confounders under observational data (Huang et al., 2025b; Zhang et al., 2025a). In sequential modeling, **knowledge-guided, semantically consistent contrastive learning** enforces augmentation-invariant semantics to stabilize sequence encoders (Shi et al., 2025). These robustness techniques are orthogonal to our retrieve→generate→feedback design and can be composed with KDTreeRAG and R-Critic.

While KDTree-based nearest-neighbor search is a well-established tool in recommender systems, our KDTreeRAG differs in both *role* and *optimization*: it *conditions* the LLM generator with user-CF contexts and is *jointly refined online* via R-Critic's generative feedback, rather than serving as a static retriever detached from generation.

In contrast to prior work that relies on static pre-trained retrievers or fixed graph priors, our KDRAG-Critic-LLM-RS forms a closed retrieve→generate→feedback loop in which KDTreeRAG steers generation and R-Critic refines retrieval online, improving robustness under behavioral drift and cold-start conditions (Huang et al., 2025a; Zhang et al., 2025d).

2.2. LLMs with RAG

In recent years, integrating external retrieval signals into large language model (LLM)-based recommender systems has become a research hotspot, with Retrieval-Augmented Generation (RAG) methods playing a crucial role. RAG improves recommendation accuracy, diversity, and personalization by retrieving auxiliary information-such as similar user behaviors-from external databases and injecting it as contextual cues into the LLM input (Chen et al., 2018; Zhang et al., 2016).

Early RAG (Retrieval-Augmented Generation) methods were primarily applied to question answering and text generation tasks, typically enriching prompts by retrieving knowledge snippets or user reviews (Jeong et al., 2024; Li et al., 2024a; Shi et al., 2023; Zhang et al., 2025c). Although some studies have further introduced structured data to enable adaptive modeling of user preferences (Islam et al., 2024), or designed bidirectional feedback mechanisms between retrieval and generation to improve retrieval precision (Wang et al., 2023), significant challenges remain in effectively applying retrieval mechanisms to personalized recommendation.

To address these issues, this paper proposes the KDRAG-Critic-LLM-RS framework, which integrates a user collaborative filtering-based efficient KDTree retrieval-augmented generation module (KDTreeRAG) with a project collaborative filtering-based generation feedback optimization module (R-Critic), forming a closed-loop collaborative system between retrieval and generation. This framework not only improves retrieval efficiency and stability but also continuously optimizes retrieval strategies through generation feedback, enabling rapid response to dynamic changes in user interests, enhancing the personalization, accuracy, and robustness of recommender systems.

2.3. LLM with feedback

In recent years, researchers have explored providing textual feedback on generated responses, enabling large language models (LLMs) to iteratively refine their outputs based on this feedback (Akyürek et al., 2023; Lee et al., 2023; Schick et al., 2022; Yu et al., 2023). These approaches help improve response quality and reduce hallucinations by fine-tuning model outputs. Early work on critic-style feedback primarily relied on costly human annotations, which limited scalability and generalization. To address this, some methods employ critique models that automatically generate feedback (Akyürek et al., 2023; Schick et al., 2022). These models are typically trained via supervised fine-tuning or reinforcement learning using gold-standard critiques collected from online forums or Wikipedia edit histories, often leveraging smaller LLMs. However, in specialized domains, such datasets are often unavailable or

expensive to create. Other approaches utilize automated reasoning engines as critics to bolster LLMs' logical reasoning capabilities (Kalyanpur et al., 2024). Additionally, some research prompts LLMs to interact with external tools-such as search engines and code interpreters-to verify key aspects of initial outputs and revise them accordingly based on verification feedback?

Building on the practical needs of recommender systems, this study proposes the KDRAG-Critic-LLM-RS framework, which innovatively incorporates an external critique feedback mechanism. It combines a generation scoring module based on item collaborative filtering (R-Critic) with an efficient retrieval-generation module based on user collaborative filtering (KDTreeRAG) to form a closed-loop retrieval-generation-scoring-feedback system. This system dynamically optimizes recommendation results, enhancing the accuracy, personalization, and stability of generated recommendations. It effectively addresses issues of scarce feedback data, limited feedback mechanisms, and poor domain adaptability found in traditional methods, significantly improving the practical value and robustness of recommender systems.

3. Problem formulation

In the recommendation task based on Large Language Models (LLM), the goal is to generate personalized recommendation lists for each user based on their historical interaction data. Let the user set be $\mathcal{U} = \{u_1, u_2, \dots, u_m\}$, where each user u_i has an interaction history H_{u_i} that represents the interactions between the user and items in the item set $\mathcal{I} = \{i_1, i_2, \dots, i_n\}$. The interaction data can include ratings, clicks, views, etc. This interaction data is key to understanding the user's interests and preferences.

Each item i_j typically has some attributes (such as title, category, rating, etc.), which help in generating personalized recommendations. The interaction sequence for a user $S_{u_i} = \{(i_1, r_1), (i_2, r_2), \dots, (i_k, r_k)\}$ contains the interactions between the user and multiple items, where r_j represents the user's interaction with item i_j (such as a rating). These interaction sequences S_{u_i} form the foundational data for generating recommendations.

We define the LLM-based recommendation task as follows: Given the user set $\mathcal{U}$ and item set $\mathcal{I}$, the goal is to design a generative recommendation model $\mathcal{M}$ that can generate personalized recommendation lists L_{u_i} based on the user's historical interaction data H_{u_i} . This recommendation list should maximize the user's satisfaction with the recommended items and dynamically adjust based on the user's behavior data and item characteristics. Mathematically, the recommendation task can be represented as:

$$\mathcal{M} : \mathcal{U} \times \mathcal{I} \rightarrow \text{Recommendation List} \quad (1)$$

Where $\mathcal{M}(u_i)$ returns the personalized recommendation list for user u_i . The model learns the relationships between users and items to generate recommendations that match the user's interests closely.

However, the LLM-based recommendation task faces several challenges. First, user behavior data may contain noise or irrelevant information, which can affect the accuracy of recommendations. Second, LLMs are usually static and hard to update in real-time, making it difficult to adapt to changing user preferences quickly. Third, LLMs have limited input length, which makes it challenging to process users' full history or large-scale item data, potentially leading to information loss or context missing. Finally, the lack of an effective feedback mechanism means that the generated recommendations cannot be optimized based on real-time user feedback, causing the recommendation system to be less responsive to dynamic changes.

To address these challenges, this paper proposes a recommendation framework based on a retrieval-generation-feedback closed-loop mechanism, which can solve the above issues and improve the accuracy, personalization, and adaptability of recommendations to dynamic user preferences.

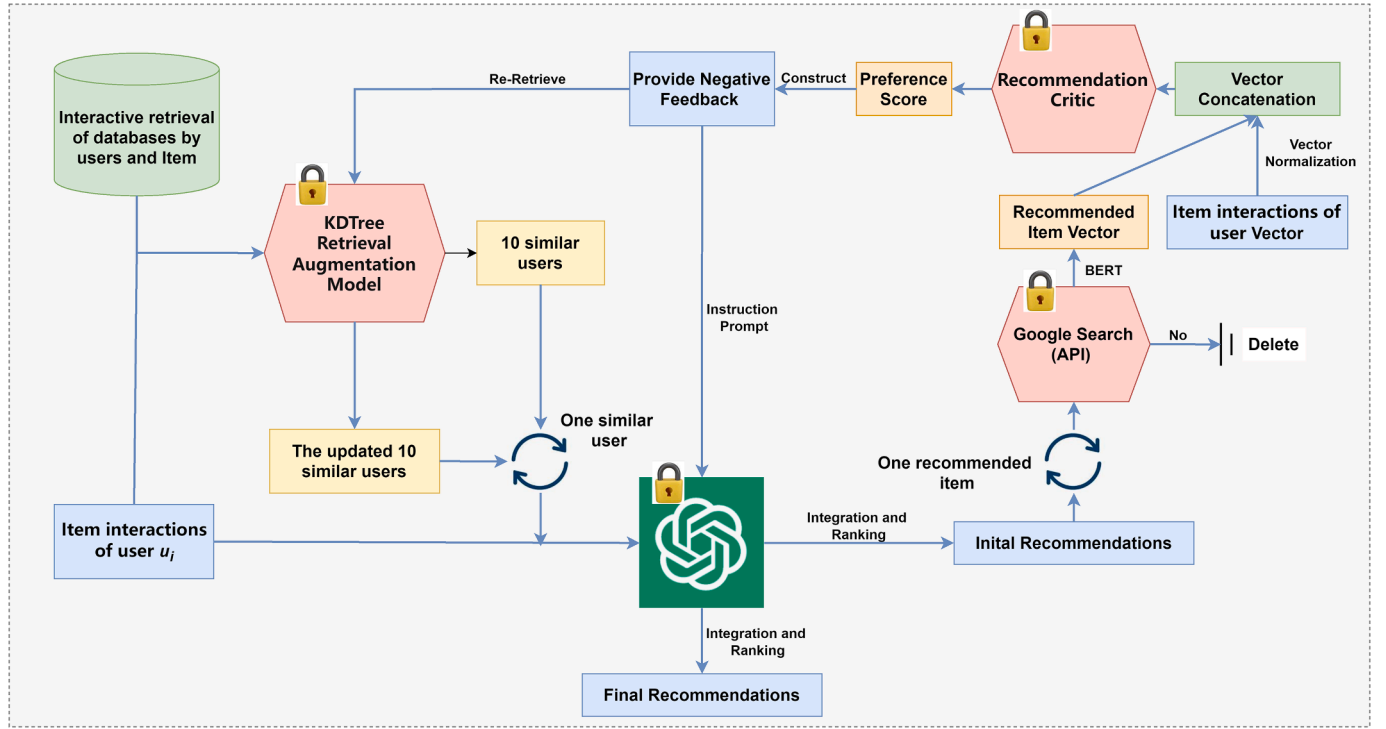


Fig. 1. KDRAG-critic-LLM-RS architecture.

4. Methodology

4.1. KDRAG-critic-LLM-RS architecture

Fig. 1 illustrates the overall architecture of KDRAG-Critic-LLM-RS, which consists of three core modules: a pretrained large language model (LLM), an efficient KDTree-based retrieval module (KDTreeRAG), and a scoring module with preference feedback (Recommendation Critic, abbreviated as R-Critic). The KDTreeRAG module is composed of a pretrained Recommendation RAG and a KDTree spatial indexing structure. The training process of KDTreeRAG is shown in Fig. 2, where the core objective is to guide the LLM in learning user collaborative filtering patterns through a closed-loop retrieve-generate-feedback mechanism, enhancing the personalization quality of generation-based recommendation. Meanwhile, the R-Critic module evaluates recommendation results from an item-based collaborative filtering perspective and participates in optimization through feedback, as shown in Fig. 8. Module icons indicate parameter states: a lock icon represents frozen parameters, while a flame icon indicates trainable ones.

The entire recommendation process consists of three stages—retrieval, recommendation, and feedback optimization—which work together to form a complete personalized recommendation loop.

In the retrieval stage, the system leverages the target user's interaction history (including movie titles and ratings) to retrieve 10 behaviorally similar users from an external user corpus using the KDTreeRAG module. Unlike traditional vector-based RAG retrieval, KDTreeRAG inherits the strong collaborative filtering modeling capabilities of RAG and integrates a KDTree-based acceleration mechanism. This significantly improves both the efficiency and accuracy of similar user retrieval at inference time, especially at scale, and provides high-quality behavioral references for subsequent recommendation.

In the recommendation generation stage, the system combines the target user's and each similar user's interaction histories to construct multiple prompt inputs in a distributed manner. These prompts are then fed into the LLM to generate candidate recommendations. To simplify the prompt structure and emphasize user preferences, only essen-

tial information such as titles and ratings is retained in the prompt (as shown in Fig. 1). Specifically, the input to the LLM includes the target user u_i 's interaction history, the interaction histories of the 10 similar users u_s , and an explicit instruction prompt. Each interaction record contains item metadata (e.g., title) and user rating (1–5 scale). For each prompt, the LLM generates 10 recommendations, resulting in a total of 100 candidate items. The system then ranks these candidates based on their frequency of occurrence across the 10 prompt outputs, selecting the top 10 most frequent items as the initial recommendation result.

In the feedback optimization stage, the R-Critic module evaluates the initial recommendation list, identifies low-quality recommendations with scores below 4, and transforms them into negative feedback expressed in natural language. This feedback is then used to constrain the next round of LLM input, guiding it to avoid such content in future generations. Simultaneously, the system re-invokes the KDTreeRAG module to retrieve a new set of similar users based on the original user history, and constructs enhanced prompts by integrating R-Critic's feedback. The LLM uses these refined prompts to generate a new set of candidate items, from which the final top 10 recommendations are selected via frequency-based ranking.

The collaboration and closed-loop integration of the modules enhance both the overall performance and stability of the recommendation system:

- The KDTreeRAG module preserves semantic representation quality while introducing an efficient spatial indexing mechanism. It enables accurate and scalable similar user retrieval from a large corpus, enriching recommendation context with more relevant and diverse behavioral samples.
- The R-Critic module effectively addresses the lack of explicit supervision in generative recommendation. By providing fine-grained scoring feedback from an item-based collaborative filtering perspective, it improves the system's ability to model user preferences, enhances controllability and robustness, and avoids the high cost of fine-tuning the LLM.

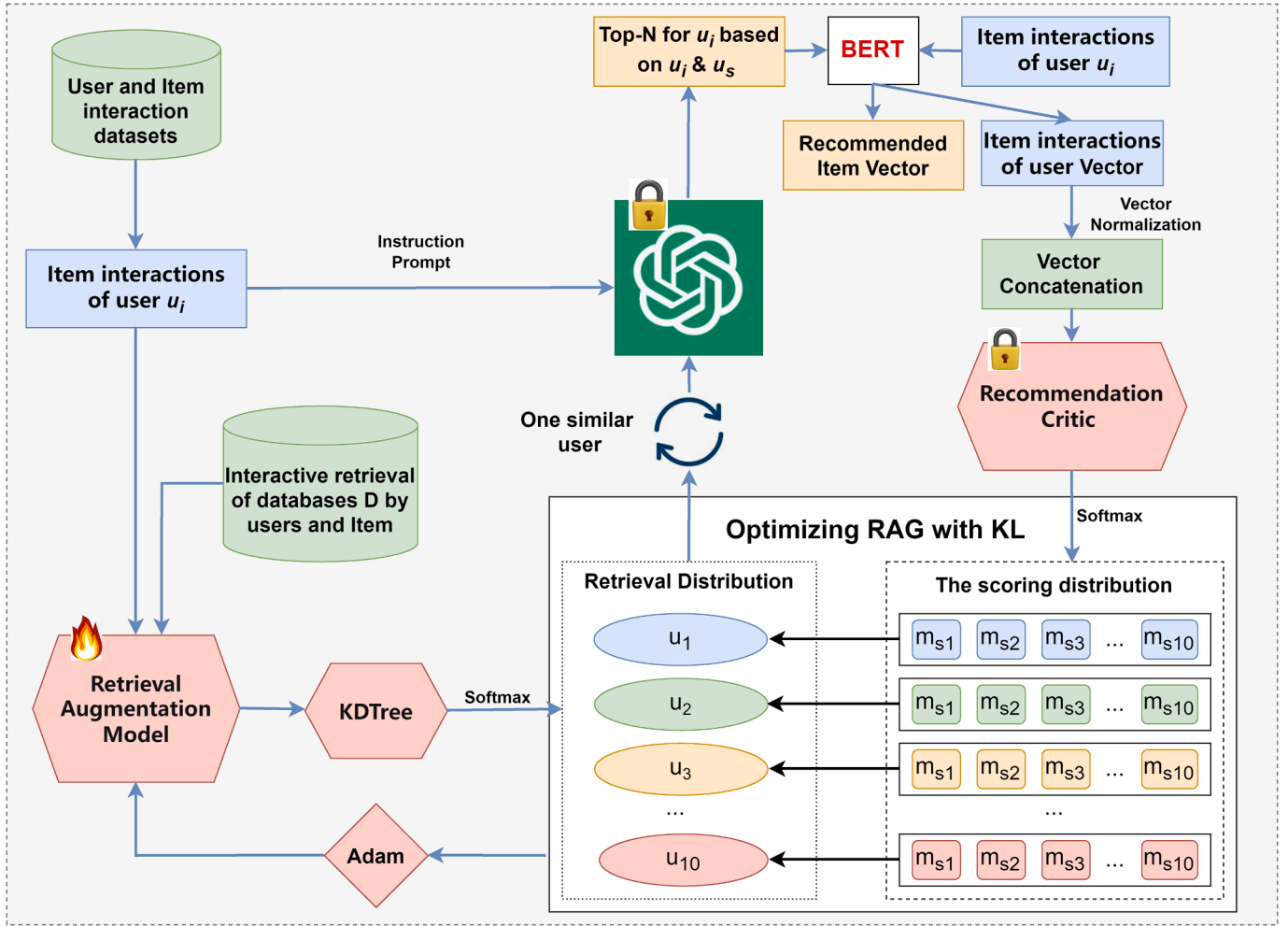


Fig. 2. KDTreerAG architecture.

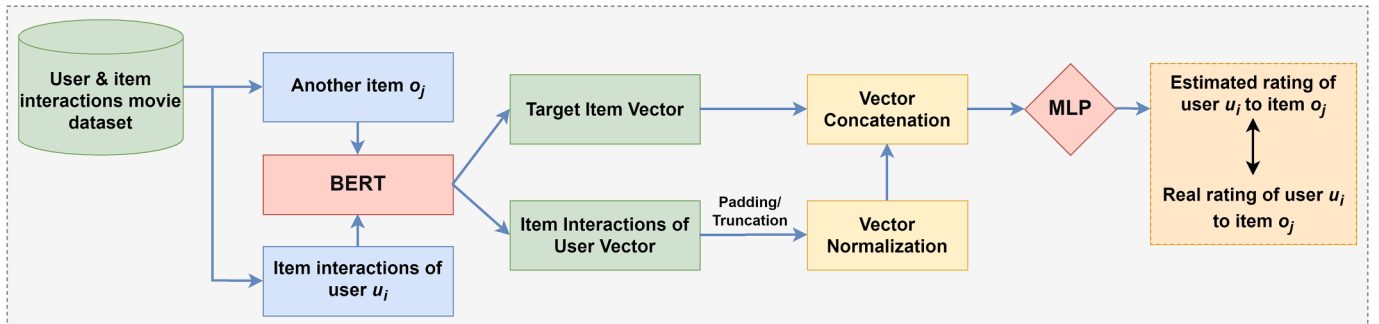


Fig. 3. R-Critic architecture.

In summary, KDRAG-Critic-LLM-RS integrates three core mechanisms—efficient user retrieval via collaborative filtering, generative personalization via language modeling, and feedback-driven optimization via item-level filtering—to construct a personalized recommendation architecture that demonstrates strong performance.

In order to better describe the KDRAG-Critic-LLM-RS model architecture, we have refined and extended Algorithm 1 to provide a detailed description of the functionality and operation process of each module.

Fig. 3

4.2. KDTreerAG: Recommendation-oriented RAG with nearest neighbor retrieval

To improve the efficiency and interpretability of retrieval in large language model (LLM)-based recommendation systems, we propose **KDTreerAG**, a recommendation-oriented Retrieval-Augmented Generation (RAG) framework enhanced with nearest neighbor search. In contrast to soft attention-based retrieval mechanisms that rely on full corpus similarity distributions, KDTreerAG explicitly selects a fixed-size

Algorithm 1 KDTree-critic-LLM-RS framework.

Require: Target user u_i , user-item interaction history D , pretrained retriever f_θ , pretrained LLM $\mathcal{G}$, R-Critic feedback module C , k (number of retrieved users)

Ensure: Top- K recommended items Λ

- 1: **Step 1: User Retrieval with KDTreeRAG**
- 2: Compute user embedding $E(u_i)$ using f_θ from user interaction history D
- 3: **for all** $u_s \in D$ **do**
- 4: Compute similarity $s(u_s, u_i) = \cos(E(u_s), E(u_i))$
- 5: **end for**
- 6: Retrieve top- k similar users $\mathcal{U}_s = \{u_1, u_2, \dots, u_k\}$ based on similarity scores
- 7: **Step 2: Generate Candidate Recommendations**
- 8: **for all** $u_s \in \mathcal{U}_s$ **do**
- 9: Construct prompt from target user u_i and similar user u_s histories (item ratings, metadata, etc.)
- 10: Generate recommendation list $\mathcal{R}_s = \mathcal{G}(\text{prompt})$ using LLM $\mathcal{G}$
- 11: **end for**
- 12: Rank the recommendations $\mathcal{R}_s$ based on frequency of occurrence across k similar users
- 13: Select top- K recommendations Λ
- 14: **Step 3: Feedback Optimization with R-Critic**
- 15: For each item $r_j \in \Lambda$, use C to estimate the rating y_j based on item and user preferences
- 16: **for all** $r_j \in \Lambda$ **do**
- 17: If $y_j < 4$, generate negative feedback using C and transform it into natural language
- 18: Use the negative feedback to refine the next set of prompts
- 19: **end for**
- 20: Reinvoke the retrieval step with feedback-augmented prompts, refine recommendations Λ' for the target user
- 21: **return** Refined top- K recommended items Λ'

set of top- k similar users based on semantic embeddings, enabling efficient and controllable retrieval. The overall architecture is illustrated in Fig. 2. Unlike prior uses where KDTree acts as a fixed similarity index, KDTreeRAG explicitly steers token-level generation and is updated by feedback signals from R-Critic, enabling responsiveness to behavioral drift and cold-start candidates produced by the LLM.

User Embedding and KDTree Retrieval. Given a target user u_i with historical interaction records H_{u_i} , we first encode the user's interaction sequence into a dense semantic embedding using a pretrained retriever model (e.g., Contriever):

$$E(u_i) = f_{\text{retriever}}(H_{u_i}), \quad E(u_i) \in \mathbb{R}^d. \quad (2)$$

A KDTree index is constructed over the embedding vectors $\{E(u_j)\}_{j=1}^N$ of all users in the database. During retrieval, we identify the top- k users $\mathcal{U}_s = \{u_1, u_2, \dots, u_k\}$ whose embeddings are closest to the target user in Euclidean space:

$$\mathcal{U}_s = \text{TopK}_{\text{KDTree}}(E(u_i), k). \quad (3)$$

LLM-Based Recommendation Generation. Each similar user $u_s \in \mathcal{U}_s$ is associated with historical interactions H_{u_s} . These histories are concatenated with the target user's interaction sequence H_{u_i} and used as context input to a large language model (LLM), which generates a personalized recommendation list:

$$\mathcal{M}_s = \{m_{s1}, m_{s2}, \dots, m_{sT}\} = \text{LLM}(H_{u_i}, H_{u_s}). \quad (4)$$

Scoring and Feedback from Recommendation Critic. To assess the quality of each recommendation, we employ a pretrained scoring model f_{critic}

that estimates a numeric rating score y_{sj} for each item $m_{sj} \in \mathcal{M}_s$, reflecting how well the item matches the target user's preferences:

$$\mathcal{Y}_s = \{y_{s1}, y_{s2}, \dots, y_{sT}\}, \quad y_{sj} = f_{\text{critic}}(m_{sj}, H_{u_i}). \quad (5)$$

We then aggregate the scores $\mathcal{Y}_s$ to estimate the overall quality of recommendations derived from user u_s :

$$P_{\text{LM}}(y | u_s, u_i) = \frac{1}{Z_s} \sum_{j=1}^T y_{sj}, \quad Z_s = \sum_{j=1}^T y_{sj}. \quad (6)$$

Optimization via KL Divergence. To guide the retriever towards selecting users that lead to higher-quality recommendations, we construct a softmax-based target distribution over the retrieved users:

$$Q(u_s | u_i, y) = \frac{\exp(P_{\text{LM}}(y | u_s, u_i)/\beta)}{\sum_{u'_s \in \mathcal{U}_s} \exp(P_{\text{LM}}(y | u'_s, u_i)/\beta)}, \quad (7)$$

where β is a temperature parameter controlling the sharpness of the distribution.

We treat the KDTree retrieval distribution $P_R(u_s | u_i)$ as a uniform distribution over the top- k neighbors unless otherwise weighted. The retriever is optimized to align with the target distribution by minimizing the Kullback–Leibler (KL) divergence:

$$\mathcal{L}_{\text{KL}} = \text{KL}(P_R \| Q) = \sum_{u_s \in \mathcal{U}_s} P_R(u_s | u_i) \log \frac{P_R(u_s | u_i)}{Q(u_s | u_i, y)}. \quad (8)$$

The gradient of $\mathcal{L}_{\text{KL}}$ is propagated back through the retriever model, and the parameters are updated using the Adam optimizer. To maintain embedding freshness and retrieval consistency, the KDTree index is periodically rebuilt using the updated user embeddings.

In summary, KDTreeRAG provides an efficient and scalable retrieval solution for large-scale recommendation systems by leveraging static nearest-neighbor search over semantically meaningful user embeddings. By decoupling the retrieval process from model parameters and aligning it with training-time optimization objectives, KDTreeRAG enables real-time, interpretable, and high-quality user retrieval without the computational overhead of dense similarity computation. Its design strikes a practical balance between inference efficiency and recommendation performance, making it well-suited for deployment in industrial-scale LLM-based recommendation scenarios.

4.3. Recommendation critic

In the KDTree-Critic-LLM-RS architecture, the Recommendation Critic (R-Critic) plays a key role as a supervised evaluation component. The Recommendation Critic is a supervised learning model, as shown in Fig. 3, designed to evaluate each item in the initial recommendation list generated by the large language model (LLM). Its goal is to accurately predict the user's rating for an input item based on the user's interaction history. The training data consists of user-item interactions, where each record includes the item's relevant information and the rating given by the user (e.g., 1 to 5). Given the item interaction history of user u_i and the item o_j to be evaluated (i.e., o_j is not in u_i 's interaction history), the Recommendation Critic learns to accurately estimate the rating of user u_i on item o_j . In this work, R-Critic is trained using users' *explicit ratings* as the supervision signal.

Specifically, user u_i 's interaction history contains multiple items, each with associated metadata (for example, for movies: title, rating; for books: title, rating). The item o_j to be predicted contains similar information about a movie or book.

The textual information of items in the user's interaction history and that of item o_j are encoded using a pre-trained encoder such as BERT.¹ Each item in the interaction history is encoded as a d -dimensional embedding vector; if the user's history contains n items, the resulting embedding dimension is $n \times d$. Item o_j is also encoded as a d -dimensional

¹ <https://huggingface.co/google-bert/bert-base-uncased>

embedding. During training, embeddings of all historical items of user u_i and the embedding of item o_j are concatenated as the model's input vector. However, since different users have varying interaction history lengths, the input embedding dimension also varies, which may affect the stability and effectiveness of training.

To address this issue and improve training stability, we standardize the number of interaction items per user to a maximum length $n_{\max}$. If a user has fewer than $n_{\max}$ items, the remaining slots are zero-padded; if more than $n_{\max}$, excess items are truncated. This standardizes the input embedding dimension to $(n_{\max} + 1) \times d$, comprising embeddings for $n_{\max}$ historical items plus one embedding for the item to be predicted.

Rating prediction is carried out using a multi-layer neural network composed of fully connected layers with ReLU activation functions. The output layer maps the network's predictions into 10 discrete rating intervals (labels 0–9). During training, the model minimizes the cross-entropy loss between the predicted and true rating labels, where the ground-truth ratings are discretized according to the following rule:

$$\text{Rating Label} = \left\lfloor \frac{\text{Predicted Rating} - 0.5}{0.5} \right\rfloor. \quad (9)$$

Specifically, continuous ratings are divided into 10 discrete labels, each representing a rating interval of width 0.5. Label 0 corresponds to the interval [0.0, 0.5), label 1 to [0.5, 1.0), label 2 to [1.0, 1.5), and so forth, up to label 9, which denotes the highest interval [4.5, 5.0]. This discretization strategy enables the model to treat rating prediction as a classification task while retaining the semantic structure of the original continuous scores.

It is worth noting that the Recommendation Critic can essentially be regarded as a recommendation model. Therefore, existing recommendation algorithms can be adapted to this module to predict the rating of any item based on the textual information in the user's interaction history. As a result, even when facing entirely new items generated by a large language model (LLM), the Recommendation Critic can still estimate their ratings by analyzing both the user's historical interactions and the semantic features of the unknown items, effectively assessing the user's potential preferences.

4.4. Computational overhead

For each target user, we retrieve $k=10$ behaviorally similar neighbors and construct k personalized prompts; consequently, the generator is invoked k times per user. To control cost, our default configuration performs *one* R-Critic pass (*1-retrieve/1-critic*). Let T_{LLM} and T_{Critic} denote the average latency of one generator call and one critic evaluation, respectively. The per-user latency is

$$T_{\text{user}} \approx k \cdot T_{\text{LLM}} + r \cdot T_{\text{Critic}}, \quad (10)$$

where k is the number of retrieved neighbors per round (default $k=10$) and r is the number of critic passes (default $r=1$). For the multi-round setting with R retrieve-generate-feedback rounds (one critic per round), the latency extends to $T_{\text{user}} \approx R(k \cdot T_{\text{LLM}} + T_{\text{Critic}})$. We further report a *3-retrieve/3-critic* variant in ablations to illustrate the accuracy-cost trade-off.

4.5. Computational complexity

Let N be the number of users, d the embedding dimension, k the number of retrieved neighbors, $H_{\max}$ the capped history per neighbor, L_{ctx} the prompt context tokens, and L_{out} the generated tokens.

Indexing. Building a KDTree on fixed- d user embeddings requires $\mathcal{O}(N \log N)$ time and $\mathcal{O}(N)$ memory. In practice, we maintain index quality via periodic bulk rebuilds.

Query. For a target user, a single k NN query has expected complexity $\mathcal{O}(\log N + k)$ in low-to-moderate d ; k is a small constant (e.g., $k=10$). Notably, KDTree pruning becomes less effective in very high dimensions; in this work we constrain user embeddings to a moderate dimension (e.g.,

via mild dimensionality reduction or feature selection) prior to indexing, so the above complexity assumptions match empirical behavior.

Generation. End-to-end wall time is primarily dominated by LLM generation, with complexity

$$\mathcal{O}(k(L_{\text{ctx}} + L_{\text{out}}) \cdot C_{\text{LLM}}), \quad (11)$$

where C_{LLM} is the per-token compute constant. We bound L_{ctx} by capping per-neighbor history ($H_{\max}$) and pruning fields.

textbfFeedback. One critic forward pass plus one re-generation adds only a constant-factor overhead.

Comparison to CF. Classical MF/BPR-style collaborative filtering requires $\mathcal{O}(C \cdot d)$ to score a candidate set of size C , or $\mathcal{O}(N \cdot d)$ for full-catalog scans. Our user-user pipeline replaces full scans with $\mathcal{O}(\log N + k)$ retrieval and a token-bounded generation budget.

5. Motivation of KDRAG-critic-LLM-RS

With the growing demand for personalized and real-time recommendations, LLM-based generative recommender systems face four major challenges:

- **(1) Lack of effective integration of collaborative filtering signals:** Current LLM-as-RS approaches often overlook explicit user-user or item-item relations, relying solely on the generative power of LLMs. This limits their ability to capture latent preference patterns and weakens the personalization quality of recommendations.
- **(2) Input length limitations:** Encoding full user histories or large-scale item metadata typically exceeds the LLM's maximum token limit. This constrains contextual richness, leading to information loss and degraded recommendation accuracy.
- **(3) Static nature of pre-trained LLMs:** LLMs are expensive to re-train or fine-tune. This rigidity makes it difficult to incorporate newly observed user behaviors or item updates, limiting responsiveness to dynamic environments.
- **(4) Noisy behavioral data and lack of feedback mechanisms:** Real-world interaction histories often contain noise, inconsistencies, or outdated preferences. In the absence of a structured feedback loop, generative models can easily amplify these issues, compromising recommendation reliability and robustness.

To address these challenges, we propose a novel generative recommendation framework: **KDRAG-Critic-LLM-RS**. This framework seamlessly integrates collaborative filtering with LLM generation via two core components:

- **KDTreeRAG:** An efficient retrieval-augmented generation module grounded in user collaborative filtering. It integrates a pretrained recommendation-oriented retriever with a scalable KDTree spatial indexing structure, enabling fast and accurate retrieval of behaviorally similar users. This design enriches contextual diversity for generation while alleviating input length constraints. Additionally, by leveraging retrieval instead of retraining, the model remains adaptive to user interest drift without altering the frozen LLM.
- **R-Critic:** A lightweight, item-collaborative filtering-inspired critique module that scores LLM-generated recommendations and produces natural-language feedback. This feedback is then used to iteratively refine retrieval and prompt construction, forming a closed-loop retrieve-generate-feedback mechanism that continuously improves output quality. Importantly, this feedback-driven process helps filter noisy signals and introduces controllability into the generative pipeline.

In summary, KDRAG-Critic-LLM-RS directly targets the four major challenges in LLM-based recommender systems by: (i) enhancing collaborative signal integration through RAG and KDTree; (ii) addressing input constraints via distributed prompt construction; (iii) achieving adaptive personalization without retraining; and (iv) establishing a

generation-level feedback mechanism to denoise and optimize recommendation outputs. This approach supports efficient, robust, and real-time recommendation without any fine-tuning of the base LLM.

6. Experiment

In this section, we present the experimental setup and results used to evaluate the performance of the proposed KDRAG-Critic-LLM-RS framework. Our experiments aim to address the following research questions (RQ):

- **RQ1:** How does the KDRAG-Critic-LLM-RS framework compare to state-of-the-art LLM-based recommender systems in terms of recommendation performance and efficiency?
- **RQ2:** What is the impact of using in-candidate recommendation (with real user ratings) on the performance of the KDRAG-Critic-LLM-RS framework in comparison to other LLM-based recommender systems?
- **RQ3:** How does the integration of KDTreeRAG retrieval and R-Critic feedback enhance the performance of the GPT4o_mini-based recommendation system compared to the standard GPT4o_mini model?
- **RQ4:** Does iterative feedback in the KDRAG-Critic-LLM-RS framework lead to a significant improvement in recommendation quality, and how many feedback loops are necessary for optimal performance?
- **RQ5:** How does the R-Critic module compare to using LLM as the critic in terms of improving recommendation accuracy, personalization, and robustness?
- **RQ6:** Comparison of KDTreeRAG and RAG for User Retrieval: How does the integration of KDTreeRAG retrieval compare to the standard RAG method in terms of retrieval speed, accuracy, and recommendation performance?
- **RQ7:** What are the individual contributions of the retrieval (KDTreeRAG) and feedback (R-Critic) components in the overall performance of the KDRAG-Critic-LLM-RS framework?
- **RQ8:** How does the integration of KDTreeRAG retrieval and R-Critic feedback enhance the quality of recommendations in real-world scenarios, as illustrated in the case studies on movie recommendations?
- **RQ9:** What is the computational time cost of KDRAG-Critic-LLM-RS?

We evaluate the performance of KDRAG-Critic-LLM-RS against state-of-the-art LLM-as-RS methods. Experiments are conducted on a cluster with 8-core CPUs, 64 GB RAM, and NVIDIA GeForce RTX 3090 GPUs. We adopt **Vicuna-1.5 (7B)** (Zheng et al., 2024a) as the backbone LLM throughout the experiments. Additionally, we extend our framework to **GPT4o_mini**, a proprietary large-scale model, to further demonstrate the model's scalability and practical applicability in black-box LLM settings. The code will be publicly released upon acceptance.

For the evaluation, we use two datasets: Movie and Book. The Movie dataset is derived from MovieLens (2021) (Kotkov et al., 2021), while the Book dataset is from Book-Crossing (2022) (Kotkov et al., 2022). We evaluate the performance on a randomly selected subset of 1k users from each dataset, using metrics such as Precision, Hit Rate (HR), and Normalized Discounted Cumulative Gain (NDCG).

6.1. Datasets and evaluation metrics

The evaluation is conducted on two datasets: *Movie* and *Book*. The Movie dataset is derived from *MovieLens (2021)* (Kotkov et al., 2021), containing interactions between 247,383 users and 84,661 movies, including movie titles, directors, main actors, and user ratings (ranging from 0.5 to 5 with a step of 0.5). The Book dataset is from *Book-Crossing (2022)* (Kotkov et al., 2022), consisting of 350,332 users and 9374 books, where each interaction includes the title, authors, language, publication year, description, URL, and rating (ranging from 1 to 5 with a step of 1).

Table 1

Performance of Oracle trained using data of different sizes (bold values are the best and underlined values are the second best).

Model		3k	5k	10k	20k	30k
Movie	micro-P	0.6797	0.6844	0.7698	<u>0.7702</u>	0.7720
	micro-R	0.5469	0.5796	0.6892	<u>0.6903</u>	0.6913
Book	micro-P	0.7106	0.7432	0.7735	<u>0.7746</u>	0.7765
	micro-R	0.5734	0.6589	0.6919	<u>0.6931</u>	0.6952

From each dataset, 3k users are randomly selected. Their complete interaction histories (only including `title` and `rating`) are used as inputs to the KdtreeRAG module (see Fig. 2). Additionally, 10k users are randomly sampled to construct training data for the Recommendation Critic module. For each user, 20 rated items and one additional candidate item (all with `title` and `rating`) are used as input (see Fig. 8), with an 8:1:1 split for training, validation, and testing.

After training KdtreeRAG, it collaborates with the LLM to generate recommendations. For each dataset, 1000 test users (not involved in training) are randomly selected. Each test user provides 20 rated items as the interaction history, which serves as input to the KDRAG-Critic-LLM-RS system (see Fig. 1). The recommendation performance is evaluated based on the proportion of preferred items (i.e., $\text{rating} \geq 4$), using the following metrics:

- **Precision:** the proportion of preferred items in the top-N recommendations. Higher values indicate better performance.
- **HR (Hit Rate):** if at least one preferred item is present in the top-N recommendations, it is counted as a hit ($HR = 1$); otherwise, it is a miss ($HR = 0$).
- **NDCG (Normalized Discounted Cumulative Gain):** measures the ranking quality of preferred items. Let $rel_i = 1$ if the item at rank i is preferred, and 0 otherwise:

$$NDCG = \frac{DCG}{IDCG}, \quad DCG = \sum_{i=1}^N \frac{2^{rel_i} - 1}{\log_2(i + 1)}.$$

Since some recommended items may lack ground-truth ratings (either they are not in the dataset or have not been rated), we adopt two evaluation strategies:

1. Restrict LLM to a candidate set with known ratings, ensuring every recommended item can be evaluated.
2. Allow the LLM to freely generate recommendations. For items without real ratings, we employ a predictive model (Oracle) to estimate ratings. Unlike R-Critic, which only uses item `title` and `rating`, Oracle leverages complete item information (e.g., authors, language, description for books; directors, actors for movies) to estimate ratings.

Although the second setting does not rely on actual ratings, the use of Oracle enables effective evaluation due to its high prediction accuracy. This demonstrates the robustness of KDRAG-Critic-LLM-RS when recommending previously unseen items.

The performance of Oracle is measured using micro-averaged precision (micro-P) and recall (micro-R), defined as:

$$\text{micro-P} = \frac{\sum_{i=1}^L TP_i}{\sum_{i=1}^L (TP_i + FP_i)}, \quad \text{micro-R} = \frac{\sum_{i=1}^L TP_i}{\sum_{i=1}^L (TP_i + FN_i)},$$

where L is the number of rating levels (10 for Movie, 5 for Book), and TP_i , FP_i , and FN_i denote the true positives, false positives, and false negatives for class i .

As shown in Table 1, Oracle achieves significant performance gains when training data increases from 3k to 10k users. Beyond 10k, improvements saturate, indicating 10k samples are sufficient for training a high-performance predictor. In our experiments, Oracle is trained using 30k users.

6.2. Baselines

The baselines include the state-of-the-art LLM-as-RS. As discussed in related work, these methods rely on the inherent, pre-trained prowess of LLMs to directly craft recommendations via meticulously designed prompts.

- Llama4Rec (LLA-RS) (Luo et al., 2024) integrates LLMs with traditional recommenders using data and prompt augmentation to combine their strengths.
- InteraRec (IR-RS) (Karra & Tulabandhula, 2024) utilizes Multimodal LLMs (MLLMs) by combining visual (e.g., posters) and textual information from user history to generate more accurate recommendations.
- GENREC (GEN-RS) (Ji et al., 2024) generates personalized recommendations by leveraging tailored prompts to capture users' historical interactions and preferences.
- GPTrec (GPT-RS) (Petrov & Macdonald, 2023) aims to tackle the challenges of sequential recommendation.
- Zero-shot-Prompt LLM-RS (ZSP-RS) (Wei et al., 2021) utilizes natural language commands to help understand complex user needs without targeted training.
- Zero-shot-CoT LLM-RS (ZSC-RS) (Kojima et al., 2022) improves reasoning over complex user preferences by incorporating a "think step-by-step" prompt into the recommendation process.
- DRDT (Wang et al., 2023) adopts a RAG-style retriever to find behaviorally similar items, then prompts LLMs to capture key user preferences and generate recommendations.
- GaCLLM (GLLM) (Du et al., 2024) integrates GCNs and LLMs to fuse structured and textual data into a unified embedding space for efficient personalized recommendation via matching scores.
- LLM-KM (LLMKMeans) employs K-means clustering to group users in the embedding space and selects cluster-level neighbors as similar users. These neighbors' interaction histories are then incorporated into LLM prompts to generate recommendations, serving as a heuristic user-based retrieval baseline.
- LLM-BTE (LLMBallTree) uses BallTree-based nearest neighbor search to retrieve semantically similar users based on user embeddings. It enhances the prompt input for LLM-based recommendation by leveraging tree-structured spatial indexing without learning-based retriever optimization.
- LLM-FT-RS (LLM-FT) fine-tunes an open-source large language model (LLM) using LoRA and user interaction data to learn the ability to generate personalized recommendations. The model is prompted with user history and generates preferred items conditionally, with a particular focus on modeling highly-rated items.
- Adaptive-RAG-LLM-RS (ARAG-RS) (Jeong et al., 2024) dynamically selects retrieval and generation policies for LLMs via a query complexity classifier to retrieve relevant item attributes and improve recommendations.
- LameR-LLM-RS (LRAG-RS) (Shen et al., 2023) enhances recommendation effectiveness by retrieving item-related information from external documents, providing richer contextual data for the LLM.

We employ a popular open-source large language model called Vicuna (Zheng et al., 2024a). Specifically, we use version 1.5 of the model with a size of 7B parameters.

6.3. RQ1: Comparison with state-of-the-art LLM-based recommender systems

To comprehensively evaluate the effectiveness of the KD-RAG-Critic-LLM-RS framework, we conduct systematic comparisons against multiple categories of baselines. Methodologically, this work integrates retrieval-feedback-prompting, enabling the LLM to incorporate both user-based and item-based collaborative filtering capabilities. To fully exploit the LLM's strengths in semantic modeling and reasoning, user

histories and item-side information are provided in textual form, yielding an end-to-end pipeline of retrieval, generation, and re-ranking. Given the fundamental differences in modeling objectives, input modality, and inference mechanism from traditional collaborative filtering—primarily reliant on numeric interaction matrices and vector operators—and other non-LLM methods, we do not perform one-to-one comparisons with non-LLM collaborative approaches. Instead, under a text-semantic setting, we focus on end-to-end performance and ablation studies to quantify the gains of the proposed method in semantic retrieval and generation. The quantitative results for the Book and Movie datasets are reported in Tables 2 and 3, respectively. Our comparisons are organized along five complementary perspectives.

(1) *Comparison with LLM-as-RS Baselines.* We compare our framework with representative large language model-based recommender systems, including GPTrec, GENREC, InteraRec, LLM-RS, ZSP-RS, and ZSC-RS. These methods typically rely on prompt design and pretrained knowledge to generate recommendations in a zero-shot or few-shot manner. However, they do not incorporate explicit collaborative filtering signals or feedback mechanisms. As shown in Tables 2 and 3, our method consistently outperforms these baselines across all metrics, indicating the effectiveness of integrating retrieval augmentation and structured feedback.

(2) *Comparison with Alternative User Retrieval Strategies.* To assess the impact of the retrieval component, we introduce two heuristic baselines: LLM-KM, which retrieves similar users using K-means clustering in the embedding space, and LLM-BTE, which uses BallTree for approximate nearest neighbor retrieval. Both methods integrate retrieved user histories into LLM prompts but lack feedback-based refinement and retrieval optimization. Our KDTreeRAG module achieves better performance, demonstrating the benefit of dynamically structured and efficiently searchable retrieval.

(3) *Retrieval-Enhanced Generation vs. Supervised Fine-Tuning.* We compare with LLM-FT, a supervised baseline that fine-tunes the LLM using explicit user-item interactions. While this method supports personalization through parameter updates, it incurs high computational cost and lacks generality. In contrast, our method delivers superior or comparable results without any fine-tuning of the LLM, highlighting the advantages of retrieval-enhanced generation and plug-and-play critic modules in terms of both performance and deployment efficiency.

(4) *Behavior-Based vs. Item-Based Retrieval.* We further examine the impact of retrieval signal types by comparing our behavior-based user retrieval strategy (KDTreeRAG) with ARAG-RS, which relies on item metadata, and RRAG-RS, which uses item co-occurrence patterns. Although both baselines enrich the LLM input via item retrieval, they fail to capture high-level collaborative user behavior. Our method yields better alignment with user preferences, verifying that user-based retrieval provides more informative and personalized context.

(5) *Comparison with LLM-RS (Vanilla LLM Recommendation).* To isolate the contribution of retrieval and feedback, we compare with LLM-RS, which directly uses the raw user history (limited to 20 items) to prompt the LLM for recommendations. This model reflects a common prompt-based paradigm without any external enhancement. Our approach significantly surpasses LLM-RS, especially in terms of NDCG and Hit Rate, due to its enriched input construction and iterative feedback adjustment.

Summary. Overall, the results in Tables 2 and 3 demonstrate that KD-RAG-Critic-LLM-RS consistently achieves state-of-the-art performance across diverse baseline categories. By combining efficient user-based retrieval, large language model generation, and feedback-driven refinement, our framework forms a retrieval-generation-feedback loop that enhances recommendation accuracy, personalization, and robustness without requiring LLM fine-tuning.

Table 2
Evaluation on book with oracle.

Model	PR@10	HR@10	NDCG@10	PR@5	HR@5	NDCG@5	PR@3	HR@3	NDCG@3
LLA-RS	0.5274	0.7360	0.6648	0.5610	0.7180	0.6214	<u>0.5737</u>	0.6860	0.6053
IR-RS	0.5177	0.6970	0.6171	0.5298	0.6750	0.5615	0.5393	0.6450	0.5501
GENREC	0.2055	0.4270	0.3877	0.2478	0.4140	0.3688	0.2790	0.3990	0.3638
GPTREC	0.5156	0.7580	0.6301	0.5218	0.7190	0.5577	0.5197	0.6650	0.5333
ZSP-RS	<u>0.5668</u>	<u>0.7650</u>	<u>0.6701</u>	<u>0.5758</u>	<u>0.7230</u>	0.6165	0.5830	0.6840	<u>0.6017</u>
ZSC-RS	0.4294	0.6390	0.5536	<u>0.4524</u>	0.6100	0.4973	0.4563	0.5740	0.4780
DRDT	0.4123	0.6190	0.5358	0.4437	0.6050	0.4867	0.4517	0.5910	0.4791
GLLM	0.5106	0.7380	0.6418	0.5213	0.7210	0.6127	0.5379	0.6980	0.5921
LLM-RS	0.3634	0.5500	0.4822	0.3878	0.5270	0.4390	0.3960	0.4990	0.4258
LLM-FT	0.4269	0.7740	0.5964	0.4358	0.6990	0.4877	0.4387	0.6410	0.4570
LLM-KM	0.3511	0.5960	0.4861	0.3800	0.5540	0.4275	0.3813	0.5050	0.4052
LLM-BTE	0.3563	0.6580	0.5392	0.4142	0.6120	0.4679	0.4233	0.5690	0.4445
ARAG-RS	0.3698	0.6310	0.5131	0.4026	0.5790	0.4508	0.4077	0.5350	0.4291
RRAG-RS	0.5106	0.7380	0.6418	0.5213	0.7210	0.6127	0.5379	<u>0.6980</u>	0.5921
KCLLM-RS	0.5786	0.7950	0.6865	0.5888	0.7540	<u>0.6180</u>	0.5890	0.7080	0.5993

Table 3
Evaluation on movie with oracle.

Model	PR@10	HR@10	NDCG@10	PR@5	HR@5	NDCG@5	PR@3	HR@3	NDCG@3
LLA-RS	0.2873	0.5080	0.4129	0.2962	0.4620	0.3515	0.3093	0.4380	0.3406
IR-RS	0.2250	0.4670	0.3396	0.2316	0.4130	0.2801	0.2393	0.3610	0.2581
GENREC	0.2138	0.4240	0.3060	0.2104	0.3540	0.2444	0.2107	0.3070	0.2272
GPTREC	0.2491	0.5420	0.3863	0.2580	0.4730	0.3153	0.2637	0.4010	0.2861
ZSP-RS	0.2703	0.5980	0.4066	0.2684	0.5070	0.3119	0.2523	0.4040	0.2684
ZSC-RS	0.2591	0.5610	0.4005	0.2620	0.4870	0.3225	0.2693	0.4220	0.2950
DRDT	0.2813	0.6120	0.4165	0.2981	0.5370	0.3221	0.2720	0.4120	0.2889
GLLM	0.2636	0.5940	0.4027	0.2861	<u>0.5510</u>	<u>0.3727</u>	0.2896	<u>0.5340</u>	0.3089
LLM-RS	0.1941	0.4560	0.3206	0.1974	0.3680	0.2487	0.1987	0.3200	0.2263
LLM-FT	0.2813	0.6120	0.4165	0.2981	0.5372	0.3221	0.2720	0.4120	0.2889
LLM-KM	0.2744	0.6030	0.4172	0.2842	0.4920	0.3364	0.2873	0.4380	0.3121
LLM-BTE	0.2636	0.5880	0.4374	0.2998	0.4920	0.3550	0.3027	0.4320	0.3276
ARAG-RS	0.2533	0.5700	0.3990	0.2668	0.4660	0.3262	0.2737	0.4100	0.3060
RRAG-RS	<u>0.3127</u>	<u>0.6300</u>	<u>0.4676</u>	<u>0.3214</u>	0.5190	0.3702	<u>0.3233</u>	0.4610	<u>0.3445</u>
KCLLM-RS	0.3792	0.7660	0.6088	0.4580	0.7130	0.5342	0.4567	0.6280	0.4878

6.4. RQ2: Impact of in-candidate recommendation on performance

To rigorously evaluate the performance of KDRAG-Critic-LLM-RS under controlled conditions, we conduct a series of comparisons in the *in-candidate recommendation* setting, where all candidate items for recommendation are guaranteed to have real user ratings. This evaluation protocol enables precise computation of standard ranking metrics such as Precision, Hit Rate (HR), and Normalized Discounted Cumulative Gain (NDCG).

The in-candidate setting, though simplified compared to real-world recommendation, offers a reliable and reproducible benchmark. It allows us to isolate and examine the effectiveness of retrieval-based prompt construction, generation strategies, and feedback refinement without confounding from unrated or novel items. This setup is especially valuable for understanding how different retrieval strategies and model designs contribute to ranking performance.

We compare KDRAG-Critic-LLM-RS against a wide range of baselines from five perspectives: (1) LLM-as-RS models, (2) alternative user retrieval strategies, (3) retrieval vs. fine-tuning, (4) behavior-based vs. item-based signals, and (5) standard LLM-RS. Results are presented in Tables 4 and 5, covering both Book and Movie domains.

6.5. RQ3: Performance enhancement on large black-box models (GPT4o_mini)

To further evaluate the generalizability and scalability of our proposed modules in black-box LLM scenarios, we extend the KDTREE RAG retrieval and R-Critic feedback components to the proprietary large-scale model **GPT4o_mini**. Unlike prior experiments with open-source LLMs, GPT4o_mini is used as an immutable black-box model,

and no fine-tuning is applied. This setup reflects a practical real-world deployment condition where only prompt-level interaction is allowed.

We construct the input prompts for GPT4o_mini using the same template format described in Table 6, including user history and similar-user context retrieved by KDTREE RAG. The output is then evaluated either directly (GPT4o_mini baseline) or re-optimized through R-Critic feedback (KDRAG-Critic-GPT4o_mini-RS variant). Figs. 4 and 5 compare the performance of the two variants on the Movie and Book datasets, respectively.

In these figures, **GPT** represents the baseline GPT4o_mini-RS model without any retrieval or feedback mechanisms, while **R1-GPT**, **R2-GPT**, and **R3-GPT** represent the enhanced **KDRAG-Critic-GPT4o_mini-RS** model after one, two, and three rounds of feedback, respectively. This model integrates both user-based retrieval and item-based feedback mechanisms. The results show that in both domains, **KDRAG-Critic-GPT4o_mini-RS** achieves significant and consistent improvements in Precision, Hit Rate, and NDCG compared to the baseline GPT4o_mini-RS, with performance continuing to improve as the number of feedback rounds increases.

These findings provide strong evidence for the general applicability of our retrieval-generation-feedback framework. Specifically, they demonstrate that: (i) the KDTREE RAG module, grounded in user-side collaborative filtering, effectively enriches prompt quality for closed models; (ii) the R-Critic module introduces actionable supervision without model access; and (iii) the combined architecture significantly boosts performance even under inference-only constraints. This confirms the practical robustness and transferability of our method to black-box LLMs, making it suitable for scalable recommendation deployment in real-world systems.

Table 4

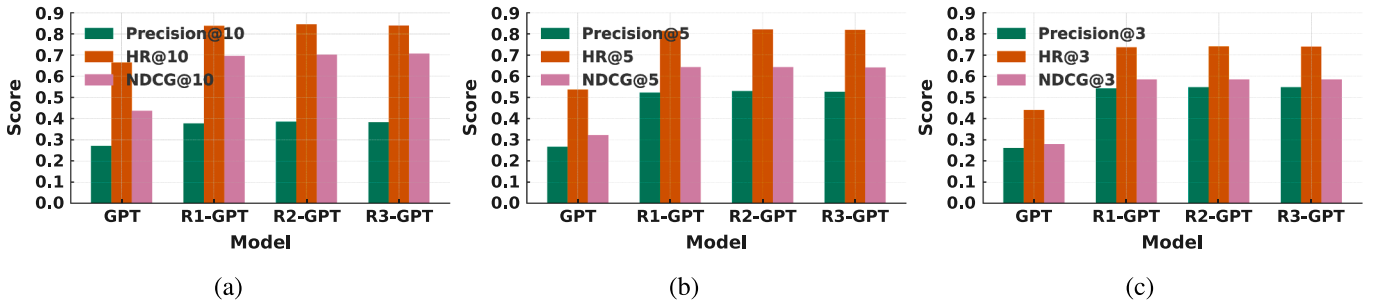
Evaluation on book with a candidate set.

Model	PR@10	HR@10	NDCG@10	PR@5	HR@5	NDCG@5	PR@3	HR@3	NDCG@3
LLA-RS	0.3669	0.6880	0.6312	0.4676	0.6840	0.6013	0.5130	0.6780	0.5914
IR-RS	<u>0.5461</u>	0.8960	<u>0.7681</u>	0.6168	0.8890	0.6904	<u>0.6420</u>	0.8710	0.6798
GENREC	0.5351	0.8780	0.7520	0.6026	0.8680	0.6696	0.6290	0.8500	0.6609
GPTRREC	0.5459	0.8950	0.7590	0.6100	0.8860	0.6797	0.6280	0.8630	0.6647
ZSP-RS	0.1387	0.6260	0.5781	0.2558	0.6260	0.5709	0.3517	0.6210	0.5551
ZSC-RS	0.1539	0.5010	0.4526	0.2642	0.5160	0.4383	0.3187	0.4920	0.4164
DRDT	0.3858	0.6790	0.5914	0.4251	0.6320	0.5867	0.4997	0.6650	0.5842
GLLM	0.4723	0.8110	0.7212	0.4923	0.7940	0.7183	0.5021	0.7850	0.6863
LLM-RS	0.1362	0.6350	0.6043	0.2292	0.6320	0.5956	0.3230	0.6290	0.5877
LLM-FT	0.3386	0.6920	0.5114	0.3068	0.6120	0.5752	0.3574	0.6170	0.5496
LLM-KM	0.3598	0.6320	0.5431	0.4026	0.5790	0.5508	0.4077	0.6490	0.5291
LLM-BTE	0.4630	0.8060	0.7012	0.4991	0.7990	0.7085	0.5106	0.7860	0.6723
ARAG-RS	0.3568	0.6720	0.6012	0.4372	0.6810	0.6122	0.5035	0.7010	0.6018
RRAG-RS	0.5051	0.8320	0.7350	0.5826	0.8310	0.6402	0.5995	0.8230	0.6419
KCLLM-RS	0.5715	0.9260	0.8065	0.6374	0.9230	0.7955	0.6440	0.9040	0.7667

Table 5

Evaluation on movie with a candidate set.

Model	PR@10	HR@10	NDCG@10	PR@5	HR@5	NDCG@5	PR@3	HR@3	NDCG@3
LLA-RS	0.1833	0.4150	0.3374	0.2268	0.4090	0.3096	0.2533	0.3910	0.2988
IR-RS	0.2802	0.5680	0.4329	0.3214	0.5500	0.3769	0.3373	0.5100	0.3611
GENREC	0.3355	0.6810	0.5543	0.3976	0.6640	0.4726	0.4210	0.6280	0.4537
GPTRREC	<u>0.3665</u>	<u>0.7120</u>	<u>0.5803</u>	0.4272	<u>0.6960</u>	0.4882	<u>0.4407</u>	<u>0.6530</u>	0.4667
ZSP-RS	0.2508	0.5480	0.4498	0.3030	0.5360	0.3935	0.3310	0.5050	0.3798
ZSC-RS	0.2671	0.5370	0.4409	0.3168	0.5190	0.3675	0.3337	0.4910	0.3531
DRDT	0.3251	0.6720	0.5323	0.3872	0.6540	0.4679	0.4011	0.6380	0.4312
GLLM	0.2853	0.5970	0.4978	0.3321	0.5760	0.4653	0.3249	0.5540	0.3647
LLM-RS	0.0790	0.3060	0.2785	0.1290	0.3040	0.2670	0.1673	0.2990	0.2586
LLM-FT	0.2739	0.5940	0.5218	0.3488	0.5880	<u>0.4994</u>	0.3530	0.5680	0.4719
LLM-KM	0.2154	0.5130	0.4674	0.2806	0.5130	0.4594	0.3107	0.5030	0.4434
LLM-BTE	0.2677	0.5860	0.5157	0.3186	0.5790	0.4905	0.3447	0.5600	0.4659
ARAG-RS	0.1992	0.4850	0.4378	0.2342	0.4830	0.4327	0.2583	0.4750	0.4186
RRAG-RS	0.2515	0.5590	0.4206	0.2789	0.5670	0.4272	0.2837	0.5570	<u>0.4918</u>
KCLLM-RS	0.4050	0.8520	0.7447	<u>0.4208</u>	0.8350	0.7331	0.5357	0.8320	0.6871

**Fig. 4.** GPT-4o_mini performance on movie dataset.

6.6. RQ4: Impact of iterative feedback on recommendation quality

The KDRAG-Critic-LLM-RS framework is inherently designed to support iterative refinement through a closed feedback loop, where the LLM's recommendations are evaluated by the R-Critic and updated accordingly. To investigate whether multiple feedback loops further improve recommendation quality, we extend the generation-critique-feedback process beyond a single iteration.

Specifically, after the initial round of recommendation generation, the R-Critic identifies suboptimal items (i.e., those predicted to receive ratings lower than 4) and produces feedback, which is then integrated into a refined prompt. This updated prompt is passed to the LLM for the next round of generation. We repeat this process for multiple loops and record the recommendation results at each step.

We evaluate the recommendations after each iteration using Precision, Hit Rate (HR), and NDCG. The results, shown in Figs. 6 and 7, illustrate the performance trajectory across different loop counts. Inter-

estingly, the performance converges quickly, with most of the improvement achieved after the first feedback loop. Subsequent iterations yield negligible or marginal gains.

This finding highlights the efficiency of our closed-loop design: the LLM can rapidly incorporate structured feedback from the R-Critic to refine its outputs. In practice, a single round of feedback is sufficient to achieve high-quality recommendations, making the system both effective and computationally efficient.

6.7. RQ5: Comparison of R-critic and LLM as critic for feedback

To evaluate the advantages of the pretrained **R-Critic** model compared to directly using the **LLM** as the Critic, we designed the following comparison experiment: **KCLLM-RS_R-Critic(KCLLM-RS-RC)** vs. **KCLLM-RS_LLM-Critic(KCLLM-RS-LC)**. In the **KCLLM-RS-RC** model, the **R-Critic** module is used to provide negative feedback. In contrast, the **KCLLM-RS-LC** model directly employs the **LLM** as the Critic, using

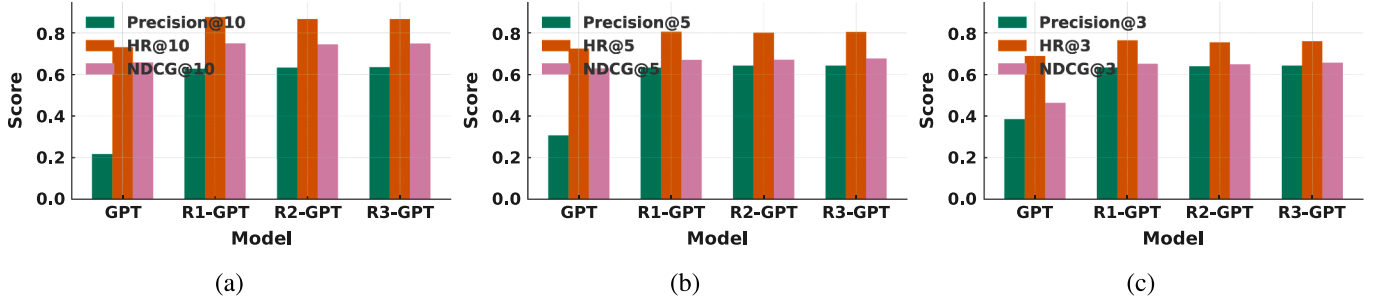


Fig. 5. GPT-4o_mini performance on book dataset.

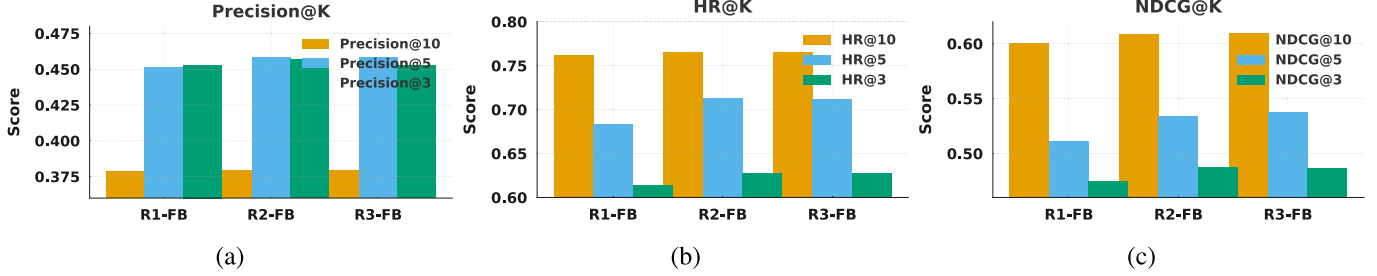


Fig. 6. Impact of iterative feedback on recommendation quality (movie domain).

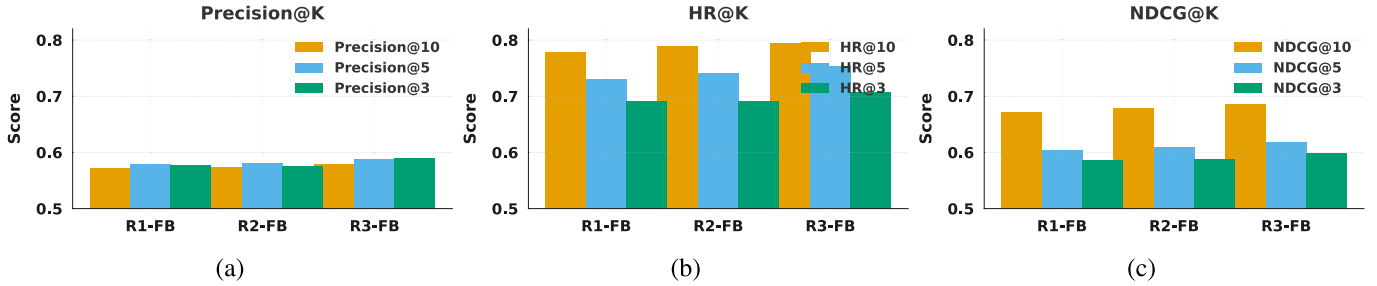


Fig. 7. Impact of iterative feedback on recommendation quality (book domain).

its generative capabilities to comment on the recommended items and provide feedback.

As shown in the experimental results in Fig. 8, the recommendation model using **R-Critic** as the feedback module significantly outperforms the model using **LLM** as the Critic across multiple evaluation metrics. Benefiting from its item-based collaborative filtering mechanism, **R-Critic** is able to provide more fine-grained and targeted feedback, effectively optimizing the generated recommendation list and guiding the model to produce higher-quality recommendations. In contrast, the **LLM**-based Critic relies solely on its own reasoning ability, which lacks specificity and effectiveness in feedback, resulting in relatively weaker performance in terms of recommendation accuracy and personalization.

These results indicate that the **R-Critic** module, with its item-based collaborative filtering feedback mechanism, can significantly improve the accuracy of generated recommendations. In contrast, the feedback mechanism of **LLM** as the Critic has limited effectiveness. Therefore, the item-based collaborative filtering **R-Critic** feedback mechanism is more effective than **LLM** as Critic in optimizing recommendations and providing more accurate and personalized recommendations.

6.8. RQ6: Comparison of KDTreeRAG and RAG for user retrieval

In order to systematically evaluate the effectiveness of the retrieval model **KDTreeRAG** compared to **RAG** in user retrieval tasks, we designed the following comparison experiment: **KCLLM-RS_KDTreeRAG** (**KCLLM-RS-KR**) vs. **KCLLM-RS_RAG** (**KCLLM-RS-R**). Where **KCLLM-RS-KR** uses the trained KDTree-based retrieval mechanism to identify

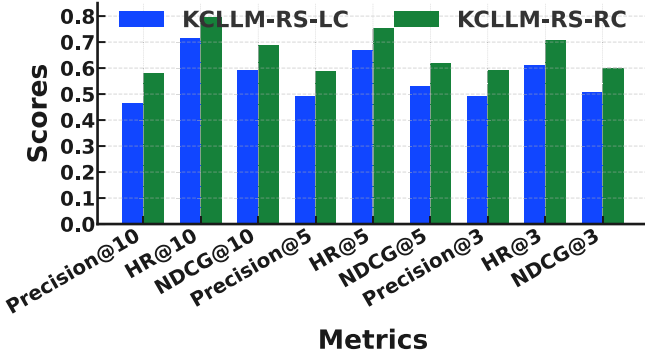
users with similar preferences to the target user, and **KCLLM-RS-R** performs similar user retrieval based on the standard RAG method.

The experimental results shown in Fig. 9 indicate that **KCLLM-RS-KR** significantly outperforms **KCLLM-RS-R** on a number of evaluation metrics (e.g., Precision_k, NDCG_k, and HR_k) related to recommendation effectiveness. This result indicates that **KDTreeRAG** performs better in the identification of potentially similar users and the modeling of synergistic contexts than the traditional RAG method, and is able to capture the synergistic signals among users more effectively, thus enhancing the ability of the LLM to model user preferences in the generation phase and further improving the accuracy and personalization of the recommendation results.

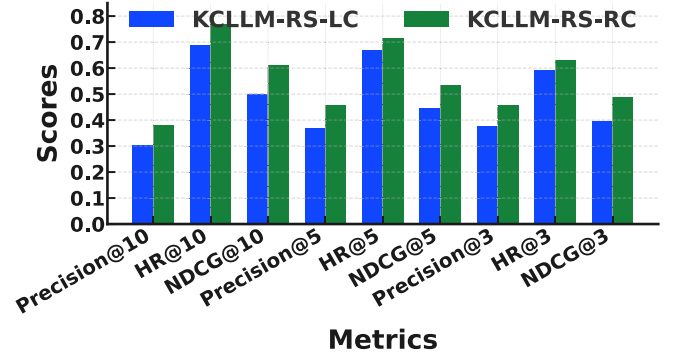
6.9. RQ7: Contribution of retrieval and feedback components

To better understand the individual contributions of each component in the **KDRAG-Critic-LLM-RS** framework, we conduct a comprehensive ablation study across five model variants, each reflecting a distinct architectural configuration. The settings are as follows:

- **KCLLM-RS_W**: A baseline model that relies solely on the large language model (LLM) to generate recommendations based on the target user's own interaction history, without any retrieval augmentation or feedback mechanisms.
- **KCLLM-RS_S**: A variant that replaces the learned KDTreeRAG retriever with a non-trainable static top-K retrieval method based on cosine similarity over frozen user embeddings. No feedback is ap-

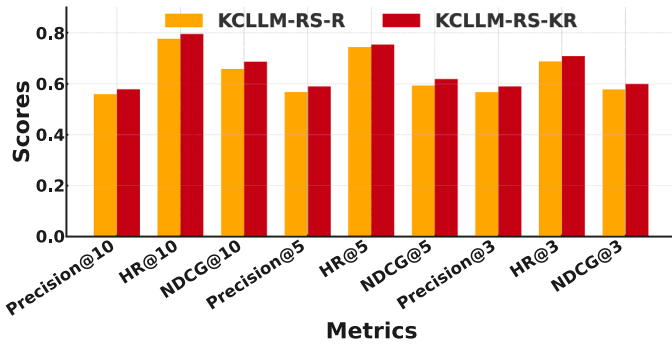


(a) Book

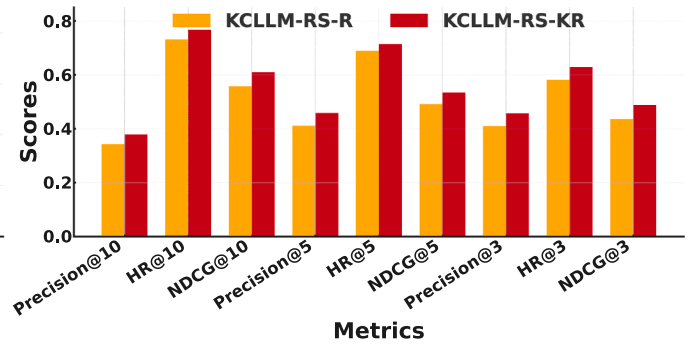


(b) Movie

Fig. 8. Comparison of R-critic and LLM as critic for feedback.



(a) Book



(b) Movie

Fig. 9. Comparison of KDTreeRAG and RAG for user retrieval.

plied. This setup tests whether heuristic user similarity alone can provide useful collaborative signals.

- **KCLLM-RS_K**: This model incorporates the full KDTreeRAG module for efficient and adaptive retrieval of similar users, enriching the prompt input to the LLM. However, it excludes the R-Critic module, removing feedback-based optimization.
- **KCLLM-RS_C**: This configuration disables the retrieval module entirely and instead applies only the R-Critic feedback mechanism. It evaluates the quality of generated recommendations and injects negative preference signals into subsequent prompts, guiding the LLM to avoid low-relevance items.
- **KCLLM-RS**: The complete framework that combines KDTreeRAG-based collaborative retrieval with R-Critic-based feedback optimization, forming a closed-loop “retrieve-generate-criticize-refine” mechanism.

Experimental results, as presented in Figs. 10 and 11, demonstrate that all variants with partial enhancements outperform the base model (KCLLM-RS_W), confirming the effectiveness of retrieval and feedback mechanisms. Notably, KCLLM-RS_K shows significant improvement over KCLLM-RS_S, highlighting the advantage of jointly optimized user retrieval over static heuristics. Similarly, KCLLM-RS_C enhances robustness by reducing exposure to low-quality outputs, although it lacks the contextual diversity provided by similar-user prompts. The full model (KCLLM-RS) consistently achieves the best performance across all metrics, validating the necessity of integrating both retrieval augmentation and feedback refinement. These findings underscore the importance of closed-loop collaborative optimization in enhancing accuracy, personalization, and system robustness in LLM-based recommendation systems.

6.10. RQ8: Case study on real-world movie recommendation scenarios

Table 6 presents the movie recommendation results for a randomly selected user (see the Experiment section for dataset details). The table also shows the prompts used in two different settings: one using a large language model (LLM) directly for recommendation, and the other combining the KDTreeRAG retrieval mechanism and the R-Critic scoring feedback module. Movies with actual user ratings are highlighted in blue, while those without real ratings are shown in black.

Compared to the baseline method (which does not use retrieval), the recommendations generated by the KDTreeRAG-Critic-LLM-RS framework demonstrate the following advantages: (1) The similar users retrieved by KDTreeRAG provide semantically relevant and preference-aligned historical information, enabling the construction of higher-quality prompt contexts; (2) The overall real ratings of the recommended movies are significantly higher; (3) The ranking of the recommendation list is more accurate, with movies having higher real and predicted ratings appearing closer to the top, thanks to the rating feedback optimization from the R-Critic module.

These results indicate that KDTreeRAG enables efficient and precise retrieval of relevant users, while R-Critic effectively guides the LLM to generate higher-quality recommendations.

6.11. RQ9: Time cost of KDTreeRAG-critic-LLM-RS

Fig. 12 presents the per-user end-to-end latency distribution with GPT-4o-mini, evaluated on 100 users (panel (a): Book; panel (b): Movie). The x-axis shows elapsed time (seconds), and the y-axis shows the number of test users for whom the final recommendation was produced; R1/R2/R3 correspond to the 1st/2nd/3rd retrieve-generate-

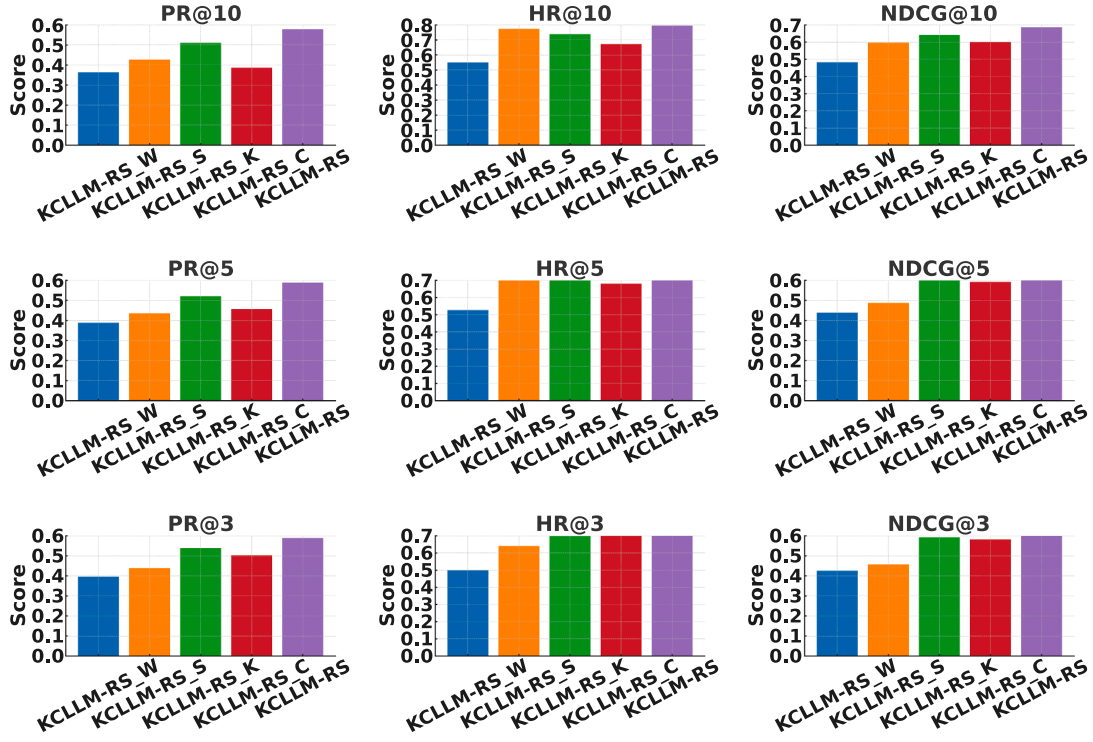


Fig. 10. Ablation results of KDRAG-critic-LLM-RS on the book dataset.

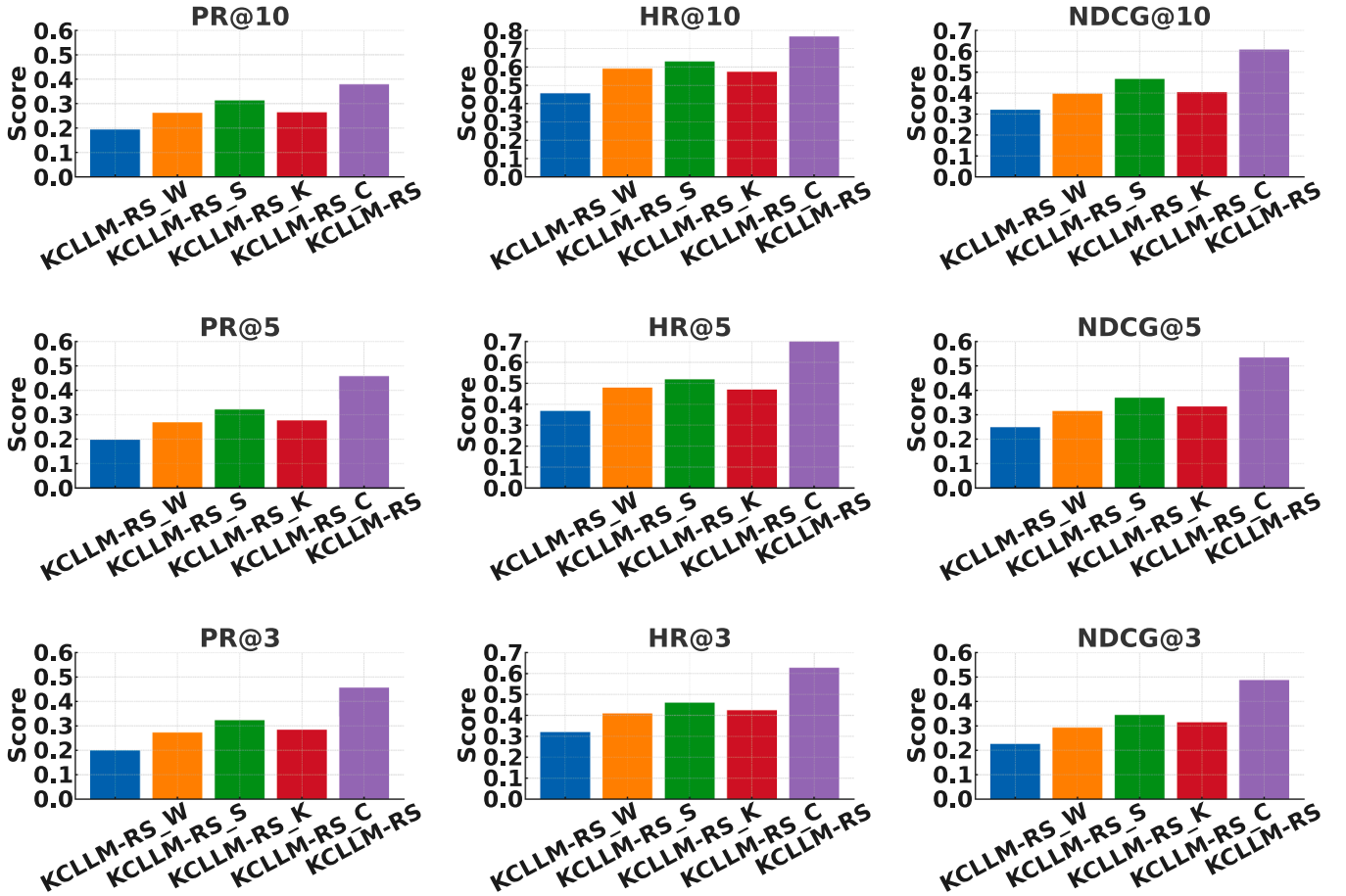


Fig. 11. Ablation results of KDRAG-critic-LLM-RS on the movie dataset.

Table 6

A case study - movie dataset - real (real rating), critic (rating estimated by the R-critic).

Prompt template for llm recommendations: "role": "system", "content": ""You are a movie recommendation system. Given a set of movies (each including the title) that a user watched, the user preference is based on the ratings that user give to these movies. Now, please recommend movies based on the user's preferences, and the recommended years for the movie are 1880 to 2021. The format for the recommended movies should be title (year)."" "role": "user", "content": ""Here are the movies that user watched and rated: < replaced by a list of movies, each with title, rating > Please recommend 10 movies and rank them according to how much the user might like them? Please base the ranking on a rating scale from 1 to 5, where 5 means I like them the most and 1 means I don't like them at all.""					
Prompt template for final recommendations: "role": "system", "content": ""You are a movie recommendation system. Given a set of movies (each including the title) that a user watched, the user preference is based on the ratings that user give to these movies. Now, please recommend movies based on the user's preferences, and the recommended years for the movie are 1880 to 2021. The format for the recommended movies should be title (year)."" "role": "user", "content": ""Here are the movies that user watched: < replace by a list of movies, each with title, rating > Here are the movie histories of a similar user (10 similar user): < replaced by the viewing history of similar users retrieved by the KDTree-RAG, include a list of movies, each with title, rating > Here are the low-quality recommended items feedbacked by the R-Critic module, containing candidates identified as not matching user preferences, used for negative guidance and optimization during the generation phase(For use in the second-round LLM recommendation): < replaced by a list of negative feedback items generated by the R-Critic module after evaluating the recommendation results, including a list of movies, each with a title and an R-Critic rating> Please recommend 10 movies based on the user's preferences and the history of similar users. Rank the movies according to how much the user might like them, on a scale from 1 to 5. A rating of 5 means the user is highly likely to enjoy the movie, while a rating of 1 means the user is unlikely to enjoy it. Take into account the low-score movies from the feedback section and avoid recommending similar movies to those with a low rating. Please provide the recommendations in order of preference, starting from the movie the user is most likely to enjoy.""					
Movies watched and rated by the user: [{"title": "Coraline (2009)", "rating": "5.0"}, {"title": "Dead Man Walking (1995)", "rating": "5.0"}, {"title": "The Perfect Guy (2015)", "rating": "4.0"}, {"title": "Run Lola Run (Lola rennt) (1998)", "rating": "3.5"}, {"title": "Gran Torino (2008)", "rating": "3.0"}, {"title": "There Will Be Blood (2007)", "rating": "4.5"}, ...]					
Initial recommendations	real	Critic	Final Recommendations	real	Critic
1 Whiplash (2014)	4.0	4.5	1 The Florida Project (2017)	5.0	5.0
2 A Beautiful Mind (2001)	-	5.0	2 The Father (2020)	-	5.0
3 The Prestige (2006)	3.5	3.5	3 Little Women (2019)	4.5	4.0
4 The Grand Budapest Hotel (2014)	-	4.0	4 Marriage Story (2019)	4.5	4.5
5 Parasite (2019)	4.5	4.5	5 Lost in Translation (2003)	4.0	4.0
6 The Pianist (2002)	-	4.0	6 Room (2015)	-	4.0
7 Dead Poets Society (1989)	-	4.0	7 The Theory of Everything (2014)	4.0	4.0
8 Her (2013)	5.0	4.5	8 Lady Bird (2017)	-	4.0
9 Arrival (2016)	-	3.5	9 The Shape of Water (2017)	3.0	3.5
10 The Farewell (2019)	-	4.0	10 Boyhood (2014)	-	3.0

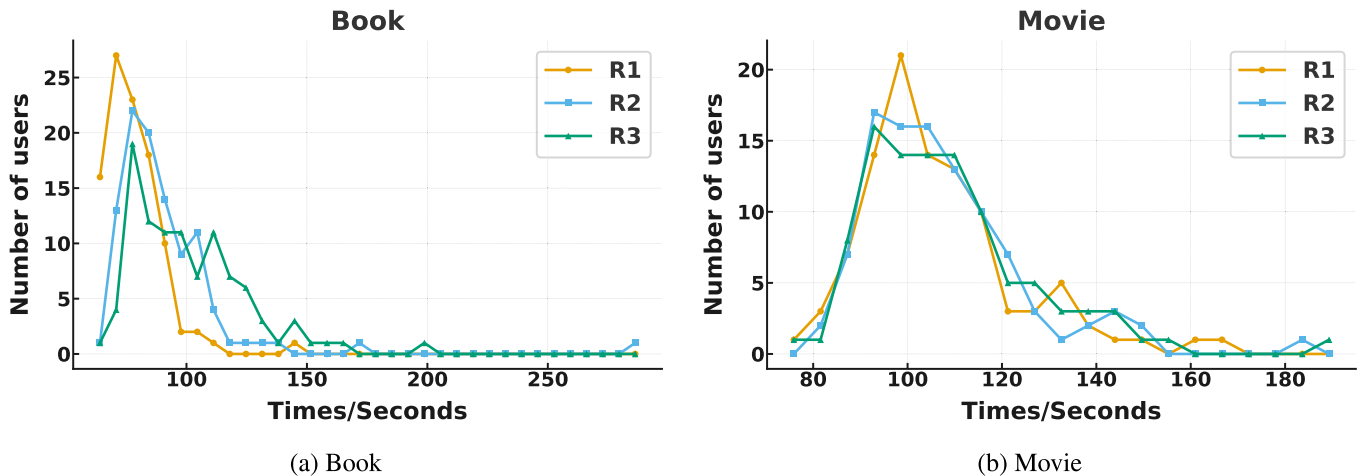


Fig. 12. Distribution of time consumed (GPT4o_mini).

feedback round, respectively. The results indicate that although multiple rounds are involved, the per-round latency distributions are highly similar; as the number of rounds increases, the total latency accumulates approximately linearly, and no additional systemic delays are observed.

7. Conclusion

This study proposes a generative recommendation framework, KDRA-G-Critic-LLM-RS, which effectively enhances LLM-based recommendation performance without fine-tuning the large language model (LLM) itself. The framework integrates a KDTREE-RAG retrieval mechanism, inspired by user-based collaborative filtering, to incorporate behavior data from similar users, and combines it with an R-Critic feedback module, inspired by item-based collaborative filtering, to provide negative item feedback signals that optimize the LLM-generated recommendations. Specifically, the KDTREE-RAG module retrieves user histories that are similar to the target user's and enriches the contextual input for the LLM. The R-Critic module evaluates the quality of the generated recommendations and offers feedback based on items the user dislikes, guiding the LLM toward more accurate and personalized recommendations. Experimental results demonstrate that this dual use of user- and item-side collaborative filtering significantly improves the recommendation system's accuracy, personalization, and adaptability. Compared with traditional methods, this framework delivers stronger performance and greater robustness without requiring any modification to the LLM parameters, offering an efficient and flexible solution for recommendation enhancement.

Limitations and Future Work. R-Critic is trained with explicit-rating supervision and currently does not incorporate implicit feedback. We plan to incorporate implicit feedback by mapping interaction events to confidence-weighted targets or by adopting BPR-style pairwise/listwise objectives with exposure correction, aiming to retain the existing pipeline while improving performance in implicit-only settings.

CRedit authorship contribution statement

Zhisheng Yang: Writing – original draft, Validation, Software, Methodology, Data curation, Conceptualization; **Li Li:** Writing – review & editing, Supervision, Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was supported by the [National Natural Science Foundation of China](#) under grant number 61877051.

References

- Akyürek, A. F., Akyürek, E., Kalyan, A., Clark, P., Wijaya, D. T., & Tandon, N. (2023). RL4F: Generating natural language feedback with reinforcement learning for repairing model outputs. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 7716–7733).
- Bao, K., Zhang, J., Zhang, Y., Wang, W., Feng, F., & He, X. (2023). TALLRec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM conference on recommender systems* (pp. 1007–1014).
- Bei, Y., Zhang, W., Wang, S., Chen, W., Zhou, S., Chen, H., Li, Y., Bu, J., Pan, S., Yu, Y. et al. (2025). Graphs meet AI agents: Taxonomy, progress, and future opportunities. *arXiv e-prints* arXiv:2506.18019.
- Boz, A., Zörgdrager, W., Kotti, Z., Harte, J., Louridas, P., Jannach, D., & Fragkoulis, M. (2024). Improving sequential recommendations with LLMs. *arXiv preprint* arXiv:2402.01339.
- Che, S., Mao, M., & Liu, H. (2024). New community cold-start recommendation: A novel large language model-based method. Available at SSRN 4828316, https://aisel.aistnet.org/aisel2024/data_soc/data_soc/6.

- Chen, R., Hua, Q., Chang, Y.-S., Wang, B., Zhang, L., & Kong, X. (2018). A survey of collaborative filtering-based recommender systems: From traditional methods to hybrid methods based on social networks. *IEEE Access*, 6, 64301–64320. <https://doi.org/10.1109/ACCESS.2018.2877208>
- Chu, Z., Hao, H., Ouyang, X., Wang, S., Wang, Y., Shen, Y., Gu, J., Cui, Q., Li, L., Xue, S. et al. (2023). Leveraging large language models for pre-trained recommender systems. *arXiv preprint* arXiv:2308.10837.
- Dai, S., Shao, N., Zhao, H., Yu, W., Si, Z., Xu, C., Sun, Z., Zhang, X., & Xu, J. (2023). Uncovering chatgpt's capabilities in recommender systems. In *Proceedings of the 17th ACM conference on recommender systems* (pp. 1126–1132).
- Du, Y., Wang, Z., Sun, Z., Chua, H., Liu, H., Wu, Z., Ma, Y., Zhang, J., & Sun, Y. (2024). Large language model with graph convolution for recommendation. *arXiv preprint* arXiv:2402.08859.
- Gong, Y., Ding, X., Su, Y., Shen, K., Liu, Z., & Zhang, G. (2023). An unified search and recommendation foundation model for cold-start scenario. In *Proceedings of the 32nd ACM international conference on information and knowledge management* (pp. 4595–4601).
- Harrison, R. M., Dereventsov, A., & Bibin, A. (2023). Zero-shot recommendations with pre-trained large language models for multimodal nudging. In *2023 IEEE international conference on data mining workshops (ICDMW)* (pp. 1535–1542). IEEE.
- Hong, M., Xia, Y., Wang, Z., Zhu, J., Wang, Y., Cai, S., Yang, X., Dai, Q., Dong, Z., Zhang, Z. et al. (2025). Eager-LLM: Enhancing large language models as recommenders through exogenous behavior-semantic integration. *arXiv preprint* arXiv:2502.14735.
- Huang, F., Bei, Y., Yang, Z., Jiang, J., Chen, H., Shen, Q., Wang, S., Karay, F., & Yu, P. S. (2025a). Large language model simulator for cold-start recommendation. In *Proceedings of the eighteenth ACM international conference on web search and data mining* (pp. 261–270).
- Huang, Z., Zhang, S., Cheng, D., Li, J., Liu, L., Lu, G., & Zhang, G. (2025b). Learning instrumental variable representation for debiasing in recommender systems. *Neural Networks*, 193, (p. 107977).
- Islam, S. B., Rahman, M. A., Hossain, K., Hoque, E., Joty, S., & Parvez, M. R. (2024). Open-rag: Enhanced retrieval-augmented reasoning with open-source large language models. *arXiv preprint* arXiv:2410.01782.
- Jeong, S., Baek, J., Cho, S., Hwang, S. J., & Park, J. C. (2024). Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. *arXiv preprint* arXiv:2403.14403.
- Ji, J., Li, Z., Xu, S., Hua, W., Ge, Y., Tan, J., & Zhang, Y. (2024). GenRec: Large language model for generative recommendation. In *European conference on information retrieval* (pp. 494–502). Springer.
- Kalyanpur, A., Saravanakumar, K., Barres, V., Chu-Carroll, J., Melville, D., & Ferrucci, D. (2024). LLM-ARC: Enhancing LLMs with an automated reasoning critic. *arXiv preprint* arXiv:2406.17663.
- Kang, W.-C., Ni, J., Mehta, N., Sathiamoorthy, M., Hong, L., Chi, E., & Cheng, D. Z. (2023). Do LLMs understand user preferences? Evaluating LLMs on user rating prediction. *arXiv preprint* arXiv:2305.06474.
- Karra, S. R., & Tulabandhula, T. (2024). InteraRec: Interactive recommendations using multimodal large language models. *arXiv preprint* arXiv:2403.00822.
- Kocof, J., Cichecki, I., Kaszyca, O., Kochanek, M., Szydio, D., Baran, J., Bielaniec, J., Gruza, M., Janz, A., Kanczler, K. et al. (2023). ChatGPT: Jack of all trades, master of none. *Information Fusion*, 99, 101861.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). Large language models are zero-shot reasoners. *Advances in Neural Information Processing Systems*, 35, 22199–22213.
- Kotkov, D., Maslov, A., & Neovius, M. (2021). Revisiting the tag relevance prediction problem. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval* (pp. 1768–1772).
- Kotkov, D., Medlar, A., Maslov, A., Satyal, U. R., Neovius, M., & Glowacka, D. (2022). The tag genome dataset for books. In *Proceedings of the 2022 conference on human information interaction and retrieval* (pp. 353–357).
- Lee, H., Phatale, S., Mansoor, H., Lu, K., Mesnard, T., Bishop, C., Carbune, V., & Rastogi, A. (2023). RL4F: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint* arXiv:2309.00267.
- Li, P., Wang, Y., Chi, E. H., & Chen, M. (2023a). Prompt tuning large language models on personalized aspect extraction for recommendations. *arXiv preprint* arXiv:2306.01475.
- Li, X., Cao, Y., Ma, Y., & Sun, A. (2024a). Long context vs. RAG for LLMs: An evaluation and revisits. *arXiv preprint* arXiv:2501.01880.
- Li, X., Chen, C., Zhao, X., Zhang, Y., & Xing, C. (2023b). E4sRec: An elegant effective efficient extensible solution of large language models for sequential recommendation. *arXiv preprint* arXiv:2312.02443.
- Li, Z., Ji, J., Ge, Y., Hua, W., & Zhang, Y. (2024b). Pap-Rec: Personalized automatic prompt for recommendation language model. *arXiv preprint* arXiv:2402.00284.
- Lin, J., Dai, X., Xi, Y., Liu, W., Chen, B., Zhang, H., Liu, Y., Wu, C., Li, X., Zhu, C. et al. (2025). How can recommender systems benefit from large language models: A survey. *ACM Transactions on Information Systems*, 43 (2), 1–47.
- Liu, C., Li, W., Li, H., Ji, R. et al. (2024). Beyond inter-item relations: Dynamic adaptive mixture-of-experts for LLM-based sequential recommendation. *arXiv e-prints* arXiv:2408.07427v1.
- Liu, H., Wei, J., Zhu, K., Li, P., Zhao, P., & Wu, X. (2025). A multi-agent reinforcement learning framework for cross-domain sequential recommendation. *Neural Networks*, 185, (p. 107192).
- Liu, J., Liu, C., Lv, R., Zhou, K., & Zhang, Y. (2023). Is chatgpt a good recommender? A preliminary study. *arXiv preprint* arXiv:2304.10149.
- Luo, S., Yao, Y., He, B., Huang, Y., Zhou, A., Zhang, X., Xiao, Y., Zhan, M., & Song, L. (2024). Integrating large language models into recommendation via mutual augmentation and adaptive aggregation. *arXiv preprint* arXiv:2401.13870.

- Petrov, A. V., & Macdonald, C. (2023). Generative sequential recommendation with GP-TRec. *arXiv preprint arXiv:2306.11114*.
- Ren, X., Wei, W., Xia, L., Su, L., Cheng, S., Wang, J., Yin, D., & Huang, C. (2024). Representation learning with large language models for recommendation. In *Proceedings of the ACM on web conference 2024* (pp. 3464–3475).
- Rong, L., Zhang, Y., Tiwari, P., & Yu, M. (2025). BegoniaGPT: Cultivating the large language model to be an exceptional k-12 english teacher. *Neural Networks*, 189, (p. 107488).
- Schick, T., Jane, A. Y., Jiang, Z., Petroni, F., Lewis, P., Izacard, G., You, Q., Nalmpantis, C., Grave, E., & Riedel, S. (2022). Peer: A collaborative language model. In *The eleventh international conference on learning representations*.
- Shen, Q., Bei, Y., Huang, Z., Zhu, J., Xu, K., Du, B., Tang, J., Jiang, Y., Huang, F., Huang, X. et al. (2025). AliBoost: Ecological boosting framework in alibaba platform. In *Proceedings of the 31st ACM SIGKDD conference on knowledge discovery and data mining v. 2* (pp. 4827–4838).
- Shen, T., Long, G., Geng, X., Tao, C., Zhou, T., & Jiang, D. (2023). Large language models are strong zero-shot retriever. *arXiv preprint arXiv:2304.14233*.
- Shen, X., Zhang, R., Zhao, X., Zhu, J., & Xiao, X. (2024). PMG: Personalized multimodal generation with large language models. In *Proceedings of the ACM on web conference 2024* (pp. 3833–3843).
- Shi, C., Yan, S., Zhang, S., Wang, H., & Lin, K.-J. (2025). Knowledge-guided semantically consistent contrastive learning for sequential recommendation. *Neural Networks*, 185, 107191.
- Shi, W., Min, S., Yasunaga, M., Seo, M., James, R., Lewis, M., Zettlemoyer, L., & Yih, W.-t. (2023). RePlug: Retrieval-augmented black-box language models. *arXiv preprint arXiv:2301.12652*.
- Tan, J., Xu, S., Hua, W., Ge, Y., Li, Z., & Zhang, Y. (2024). IdgenRec: LLM-recsys alignment with textual ID learning. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval* (pp. 355–364).
- Wang, Q., Li, J., Wang, S., Xing, Q., Niu, R., Kong, H., Li, R., Long, G., Chang, Y., & Zhang, C. (2024). Towards next-generation LLM-based recommender systems: A survey and beyond. *arXiv preprint arXiv:2410.19744*.
- Wang, X., Chen, H., Pan, Z., Zhou, Y., Guan, C., Sun, L., & Zhu, W. (2025). Automated disentangled sequential recommendation with large language models. *ACM Transactions on Information Systems*, 43 (2), 1–29.
- Wang, Y., Liu, Z., Zhang, J., Yao, W., Heinecke, S., & Yu, P. S. (2023). DrdT: Dynamic reflection with divergent thinking for llm-based sequential recommendation. *arXiv preprint arXiv:2312.11336*.
- Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., & Le, Q. V. (2021). Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*.
- Woźniak, S., Duszko, J., Kocoń, J., & Kazienko, P. (2025). Improving LLM-based recommender systems with user-controllable profiles. In *Companion proceedings of the ACM on web conference 2025* (pp. 2102–2111).
- Wu, L., Zheng, Z., Qiu, Z., Wang, H., Gu, H., Shen, T., Qin, C., Zhu, C., Zhu, H., Liu, Q. et al. (2024). A survey on large language models for recommendation. *World Wide Web*, 27 (5), 60.
- Xu, W., Yu, Z., Wang, Y., Wang, J., & Peng, M. (2024). Rs-agent: Automating remote sensing tasks through intelligent agents. *arXiv preprint arXiv:2406.07089*.
- Yu, W., Zhang, Z., Liang, Z., Jiang, M., & Sabharwal, A. (2023). Improving language models via plug-and-play retrieval feedback. *arXiv preprint arXiv:2305.14002*.
- Zhang, G., Yuan, G., Cheng, D., Liu, L., Li, J., Xu, Z., & Zhang, S., et al. (2025a). Deconfounding representation learning for mitigating latent confounding effects in recommendation. *Knowledge and Information Systems*, 67 (7), 5999–6020.
- Zhang, G., Yuan, G., Cheng, D., Liu, L., Li, J., & Zhang, S. (2025b). Mitigating propensity bias of large language models for recommender systems. *ACM Transactions on Information Systems*, 43 (6), 1–26.
- Zhang, J., Lin, Y., Lin, M., & Liu, J. (2016). An effective collaborative filtering algorithm based on user preference clustering. *Applied Intelligence*, 45, 230–240.
- Zhang, Q., Chen, S., Bei, Y., Yuan, Z., Zhou, H., Hong, Z., Chen, H., Xiao, Y., Zhou, C., Dong, J. et al. (2025c). A survey of graph retrieval-augmented generation for customized large language models. *arXiv preprint arXiv:2501.13958*.
- Zhang, W., Bei, Y., Yang, L., Zou, H. P., Zhou, P., Liu, A., Li, Y., Chen, H., Wang, J., Wang, Y. et al. (2025d). Cold-start recommendation towards the era of large language models (LLMs): A comprehensive survey and roadmap. *arXiv preprint arXiv:2501.01945*.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. et al. (2023a). Judging LLM-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 46595–46623.
- Zheng, Z., Chao, W., Qiu, Z., Zhu, H., & Xiong, H. (2024b). Harnessing large language models for text-rich sequential recommendation. In *Proceedings of the ACM on web conference 2024* (pp. 3207–3216).
- Zhu, Y., Wu, L., Guo, Q., Hong, L., & Li, J. (2024). Collaborative large language model for recommender systems. In *Proceedings of the ACM on web conference 2024* (pp. 3162–3172).